

Observe Docs Analysis Report

A competitor deep-dive to power the Observe docs revamp

Owner: Abhijai · FutureAGI

Capture date: June 16, 2026

Subjects: Braintrust · Mixpanel · PostHog · Sentry · Langfuse · FutureAGI

TL;DR — The 5 Biggest Takeaways

- **Braintrust sets the depth bar.** Every feature page pairs worked examples + parameter tables + annotated UI. We're currently ~2 levels below that. The rubric in §4 makes this measurable.
- **Langfuse wins on filters documentation.** Not because they have a single "filters" page, but because every filterable attribute (Tags, Sessions, Environments, Log Levels, Metadata) gets its own dedicated page with deep how-to content. PostHog's product analytics section has no equivalent depth. Verdict: steal the per-attribute-page model from Langfuse.
- **Mixpanel "DICs/dictionary" = Lexicon.** Confirmed. Lexicon is their events/properties data dictionary under Data Governance. Title: "Lexicon: Describe your events and data using a dictionary." The "Boards" rename (from Dashboards) and the Insights flow is documented and screenshotted.
- **PostHog's observability product is called "AI Observability"** — URL /docs/ai-observability. It is NOT called "Observe" and we did not find a legacy /llm-observability slug in any captured screenshot or navigation. The IA inside it is exemplary: Start here → Installation → Concepts → Guides → PostHog AI → Resources.
- **Our 4 biggest gaps vs Nikhil's wants:** (1) No "Getting Started" entry; (2) Troubleshooting is not last; (3) Labels violate the 2/3-word rule; (4) The filters page is 1 page of syntax — no per-filter depth.

How to Use This for the Execution PRD

This document is the complete evidence base. The Execution PRD should draw on it as follows:

Execution PRD Section	Source in This Document
New IA / sidebar order	§6 Flow Map (the proposed Observe IA) + per-subject transcriptions
Label naming rules with examples	§5 Sidebar Naming Pattern Sheet
Content depth standards	§4 Content-Depth Rubric (Braintrust bar)
Filters page spec	§5 Langfuse filters verdict + §3 PostHog evidence
Feature page presentation	§7 Feature-Presentation Pattern Library
Gap list (what to fix, in what order)	§8 Baseline Audit
Install-first ordering evidence	§4 Sentry chapter
Dashboards/Boards flow	§2 Mixpanel chapter + Boards screenshots

Reminder: What This PRD Does NOT Decide

The proposed IA in §6.3 is marked NON-BINDING. It's a first draft for the Execution PRD to react to and finalize. No actual doc pages are written or edited here.

Braintrust

Gold standard for content depth · the benchmark everything else is measured against

Sidebar — Transcribed Tree (Relevant Subtree = Full Docs)

START

Introduction
Braintrust workflow
Tracing quickstart
Evaluation quickstart
Loop agent
Plans and limits

INSTRUMENT

Overview
▶ Tracing (expandable)
User feedback
Attachments

OBSERVE

Overview
▶ Logs (expandable)
▶ Topics (expandable)
Dashboards

ANNOTATE

Overview
Human review
Custom views
Labels and corrections
▶ Datasets (expandable)
Exports

EVALUATE

Overview
Prompts
▶ Scorers and classifiers
Playgrounds
▶ Experiments
▶ Remote evals
Online scoring
Best practices

DEPLOY

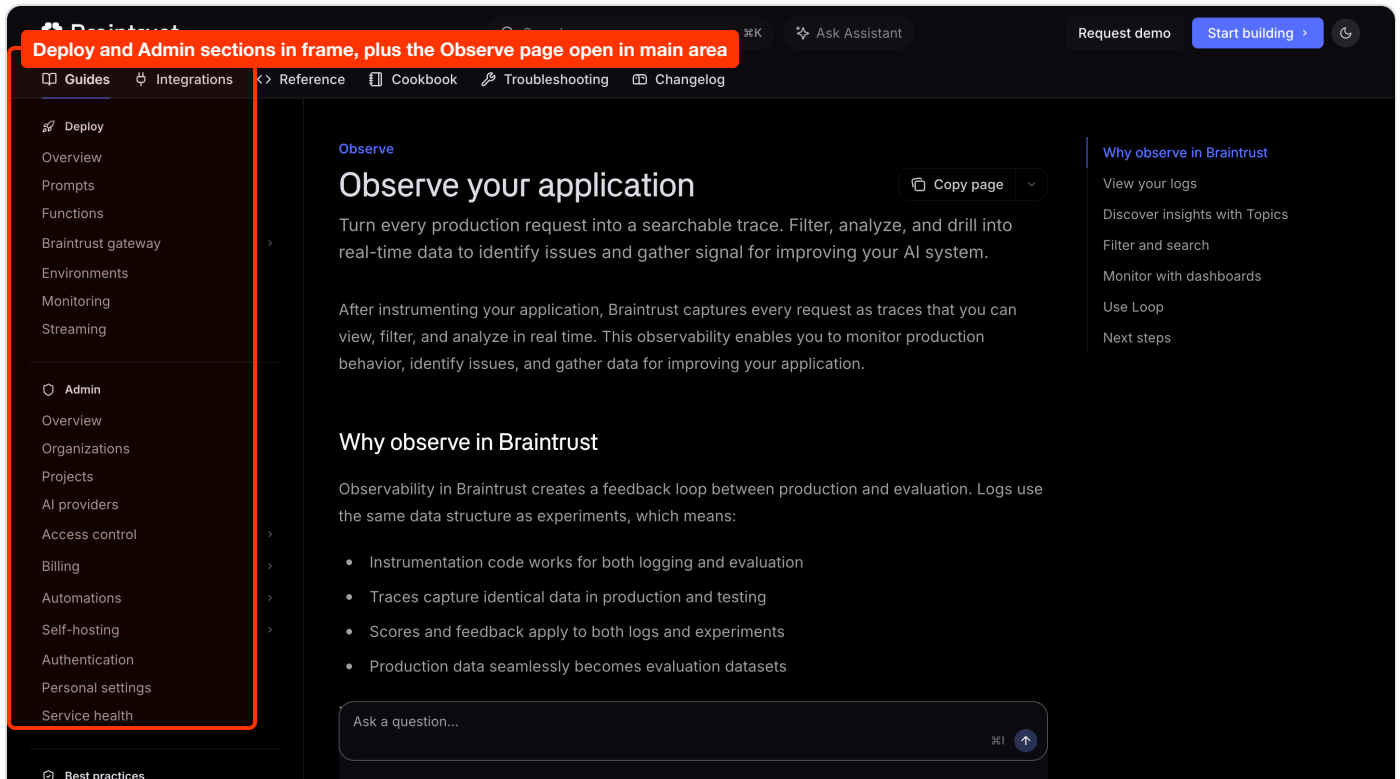
Overview · Prompts · Functions · ▶ Braintrust gateway · Environments · Monitoring · Streaming

ADMIN

Overview · Organizations · Projects · AI providers · ▶ Access control · ▶ Billing · ▶ Automations
· ▶ Self-hosting · Authentication · Personal settings · Service health

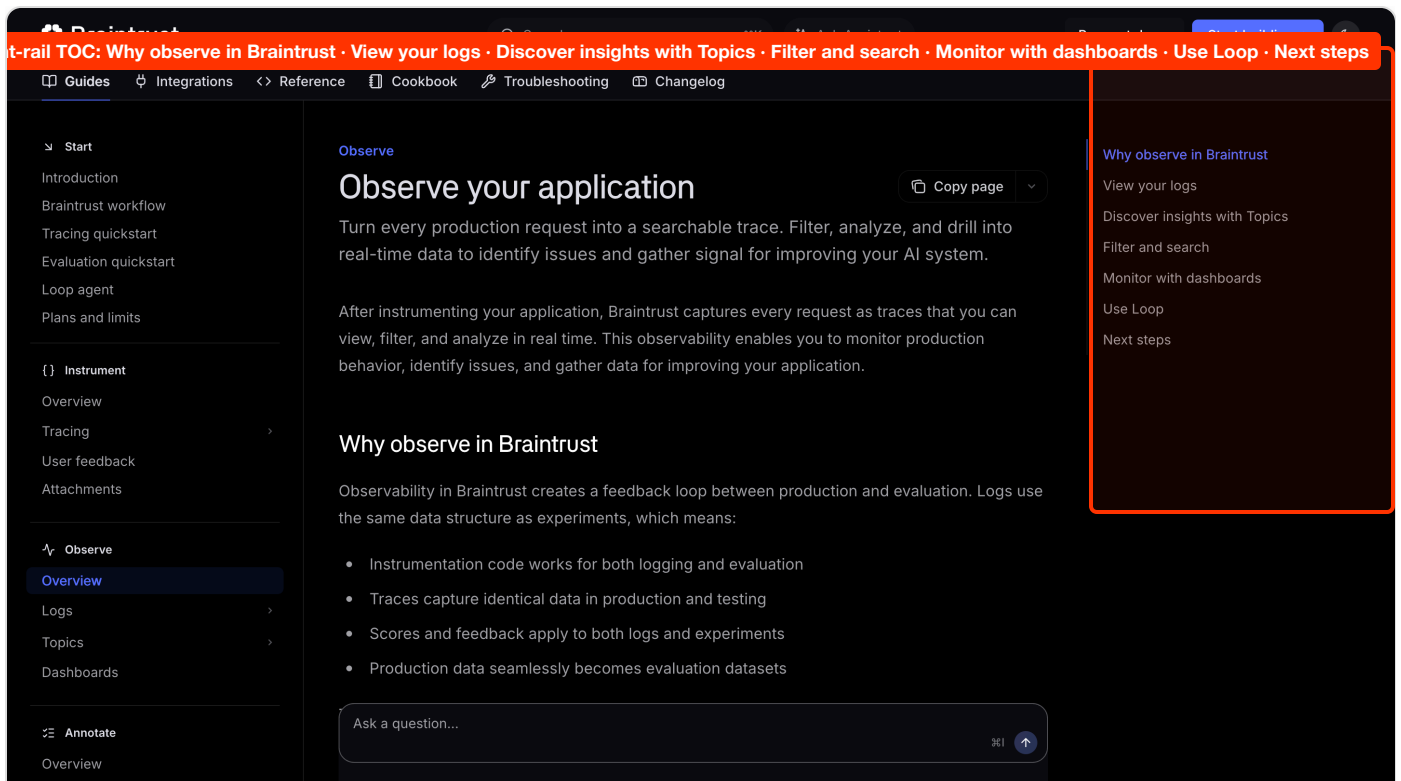
BEST PRACTICES

Evaluating agents · Writing scorers · PM workflows · Security



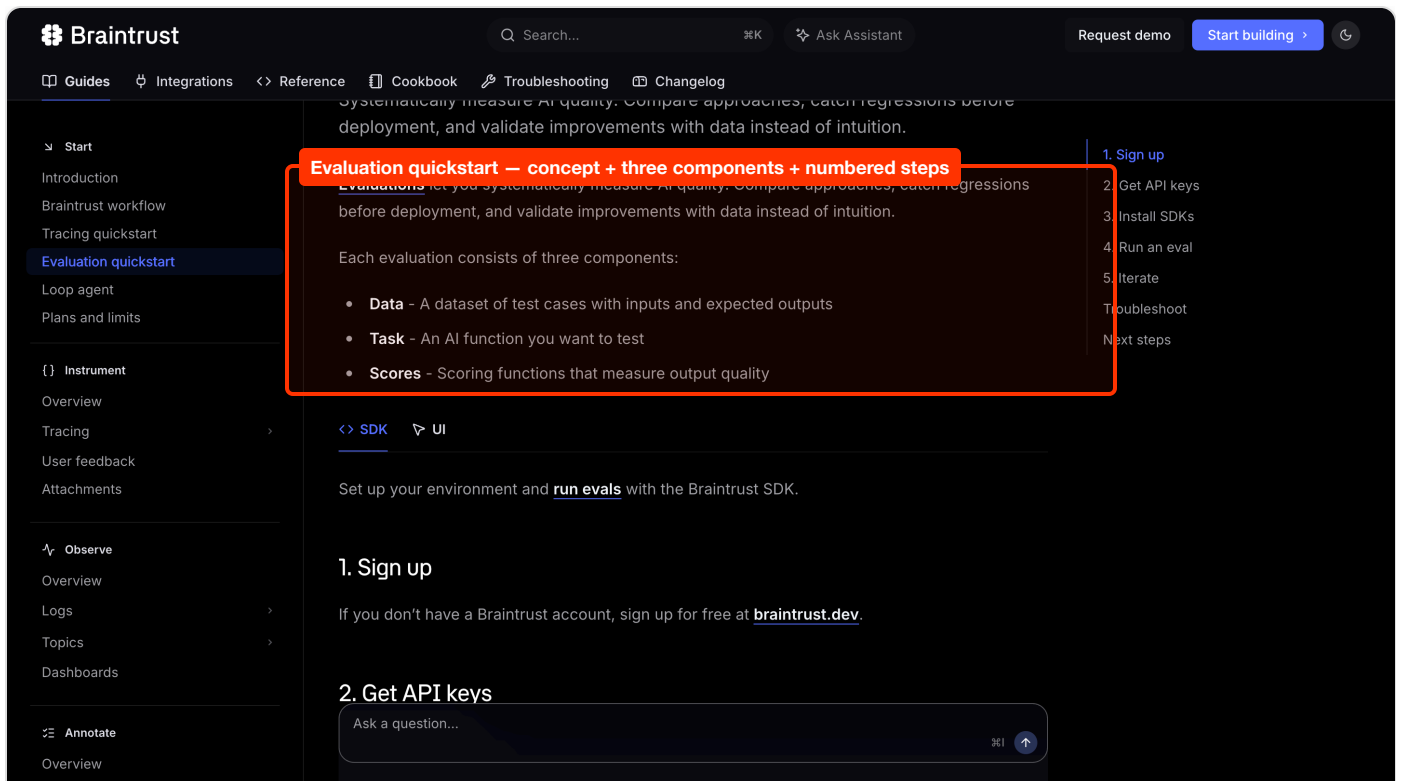
01 — Braintrust sidebar (Deploy/Admin view): this viewport shows the bottom of Braintrust's sidebar — Deploy (Overview, Prompts, Functions, Braintrust gateway, Environments, Monitoring, Streaming) and Admin (Overview, Organizations, Projects, AI providers, Access control, Billing, Automations, Self-hosting, Authentication, Personal settings, Service health, Best practices). The main area shows the Observe page with its right-rail TOC already in frame.

Even at the bottom of a long sidebar the labels stay to 1–2 words: "Overview," "Prompts," "Functions," "Monitoring," "Streaming." No four-word labels or question forms anywhere in view. The sidebar's discipline holds all the way down to the Admin section.



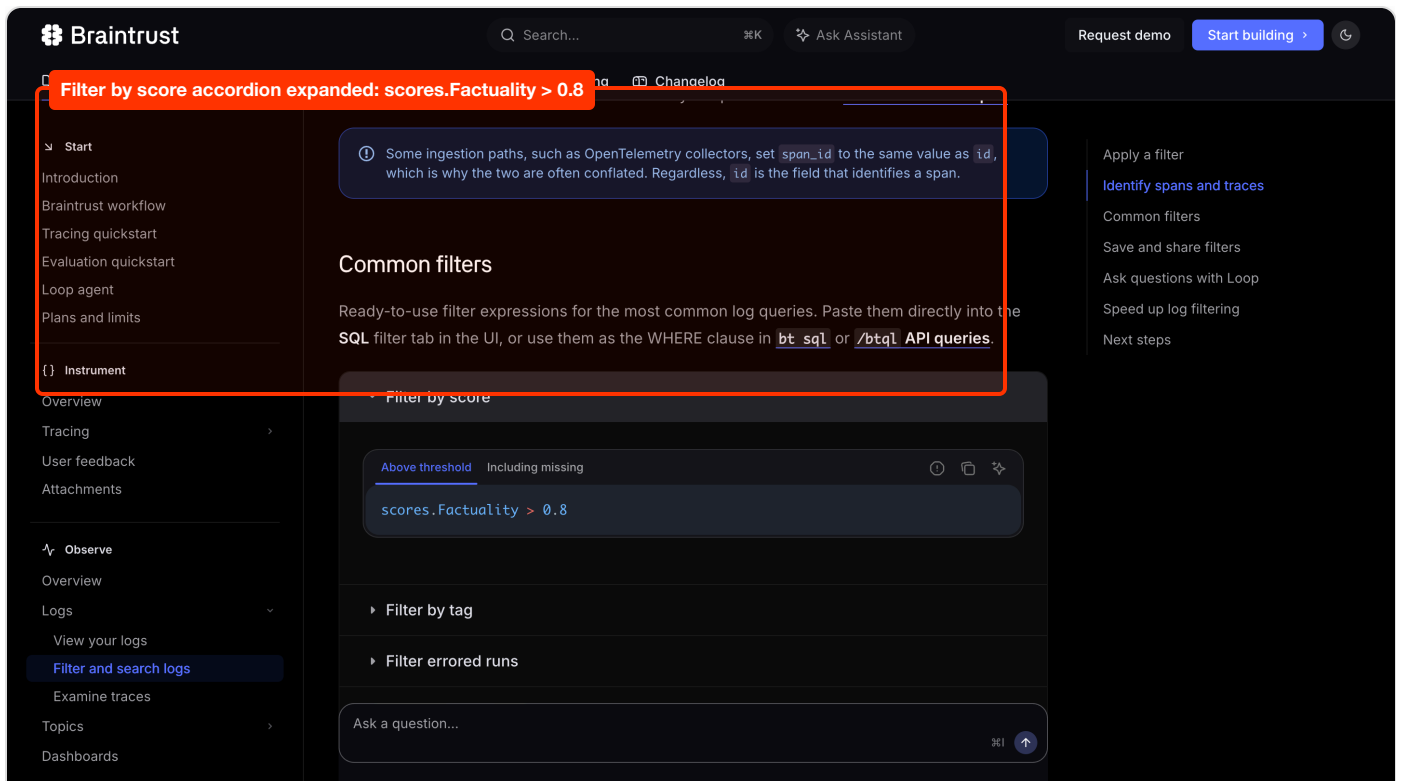
05 — Braintrust Observe section: the "Observe your application" page with its full right-rail TOC visible: "Why observe in Braintrust," "View your logs," "Discover insights with Topics," "Filter and search," "Monitor with dashboards," "Use Loop," "Next steps." The main area shows the "Why observe in Braintrust" section open with four bullet points about the feedback loop between production and evaluation.

Seven distinct sections on one page, all named like real concepts — not "Overview" filler. The right-rail TOC makes the page's scope scannable before you read a word. "Use Loop" is a specific Braintrust feature embedded right in the Observe flow, not hidden in a separate section. That integration is deliberate and it shows.



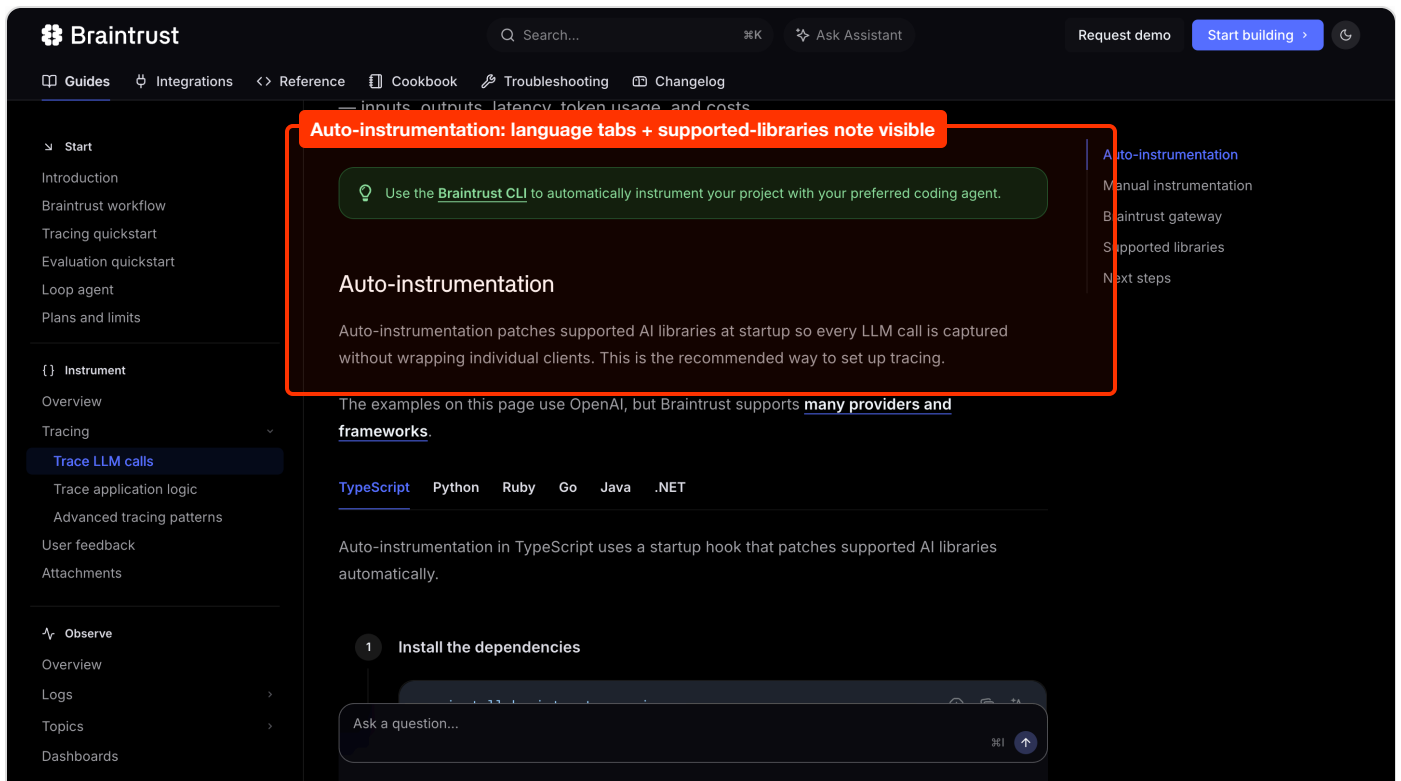
02 — Braintrust evals density: the Evaluation quickstart page opens with concept definition ("Evaluations let you systematically measure AI quality"), then immediately names the three required components — Data, Task, Scores — and transitions into a numbered step flow. Steps 1–2 are visible in this frame (Step 1: Sign up, Step 2: Get API keys); the right-rail TOC confirms the sequence continues through Step 5. The sidebar also shows the Observe section (Overview, Logs, Topics, Dashboards) confirming this sits inside a lifecycle-stage IA.

This is the triple structure that Braintrust applies consistently: you get the "why" in one paragraph, the building blocks named explicitly, and then a step-by-step you can follow. No assumption that you already know the components. We should do the same for our Evals-on-Traces page.



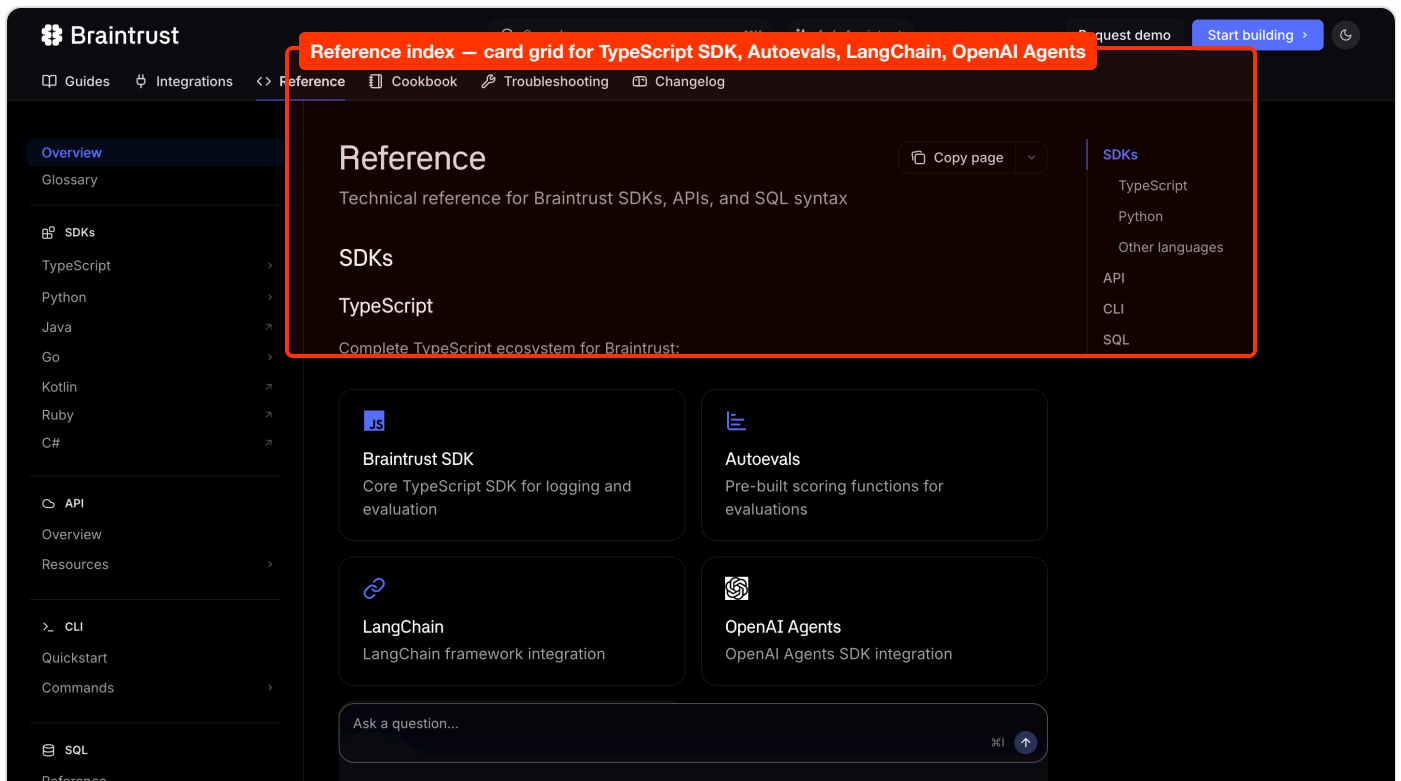
06 — Braintrust filter page: the "Common filters" page introduces itself as "Ready-to-use filter expressions for the most common log queries. Paste them directly into the SQL filter tab in the UI, or use them as the WHERE clause in bt sql or /btql API queries." The expanded "Filter by score" accordion shows the concrete expression: `scores.Factuality > 0.8`. Below it sit two collapsed accordions — "Filter by tag" and "Filter errored runs."

This is the depth we want. Each filter pattern gets its own named accordion with a copy-pasteable expression — not a prose description but the literal query. The intro makes the dual use explicit: paste into the SQL filter tab in the UI, or drop it in a bt sql WHERE clause. We have no equivalent for our trace filters right now.



03 — Braintrust tracing density: the Trace LLM calls page shows the "Auto-instrumentation" section with language tabs (TypeScript, Python, Ruby, Go, Java, .NET) and the note "The examples on this page use OpenAI, but Braintrust supports many providers and frameworks." The right-rail TOC confirms what the page covers: Auto-instrumentation, Manual instrumentation, Braintrust gateway, Supported libraries, Next steps. The sidebar shows Trace LLM calls is highlighted inside the Instrument section.

Six language tabs right at the top of the tracing page — no hunting for "does Braintrust support Python?" The "many providers and frameworks" note sets the expectation before you even see the list. Right-rail confirms this page goes beyond basics into the Supported libraries table. That breadth at a glance is the bar.



04 — Braintrust reference section: the Reference page opens as a card index: "Technical reference for Braintrust SDKs, APIs, and SQL syntax." Under TypeScript it shows four cards — Braintrust SDK (logging and evaluation), Autoevals (scoring functions), LangChain, and OpenAI Agents. The sidebar lists SDKs (TypeScript, Python, Java, Go, Kotlin, Ruby, C#), API, CLI, and SQL. This is the entry portal, not the parameter tables themselves — those are presumably one level deeper per card (not captured in this frame).

The reference entry page is a card index, not a parameter dump. Each card names what it covers — "Core TypeScript SDK for logging and evaluation" — so you know exactly which card to click before you commit. Our own Reference section has no entry page at all right now. Even a card index like this would be a major improvement.

What Braintrust Does Best

Lifecycle-stage IA with pure verb-headers. Every page pairs "what" (the concept) with "how" (the code/SQL), parameter tables, and "why this matters." Their Observe section has its own right-rail TOC. The sidebar is almost entirely 1–2 word labels — zero label violations of our 2/3-word rule.

Moves We Steal from Braintrust

- Lifecycle stage as the top-level section header (Instrument → Observe → Evaluate)
- "On this page" right-rail TOC for every feature page
- Ready-made query/filter patterns per filter type
- Prose → code → parameter table on each page (the triple structure)
- 1-word section labels: "Overview," "Logs," "Dashboards," "Topics"

Mixpanel

Best feature presentation · Boards/Insights flow · Lexicon (the "DICs/dictionary")

Relevant Subtree: Reports + Boards + Data Governance

ANALYSIS (section header)

Reports
Insights
Funnels
Overview · Quickstart · Advanced · FAQ
Retention
Flows
Apps → Impact · Signal · JQL
Boards
Sharing and Permission
Templates · Boards on Boards · Public Boards · Move Boards
Feature Flags
Experiments · Metric Trees · Users · Cohorts
Session Replay

MIXPANEL AI (section header)

Mixpanel Agent · Agentic Automations · Root Cause Analysis · MCP Server

ADMIN (section header)

Data Governance
Lexicon ← THE "DICTIONARY"
Data Views & Classification
Data Standards · Event Approval · Data Volume Monitoring

RESOLVED: What "DICs" / "dictionary" means in Mixpanel

Confirmed: Mixpanel "Lexicon" is the "data dictionary."

The page is at docs.mixpanel.com/docs/data-governance/lexicon and its title is literally: *"Lexicon: Describe your events and data using a dictionary."* The opening paragraph confirms: "Lexicon is a data dictionary that stores descriptions of events and their properties."

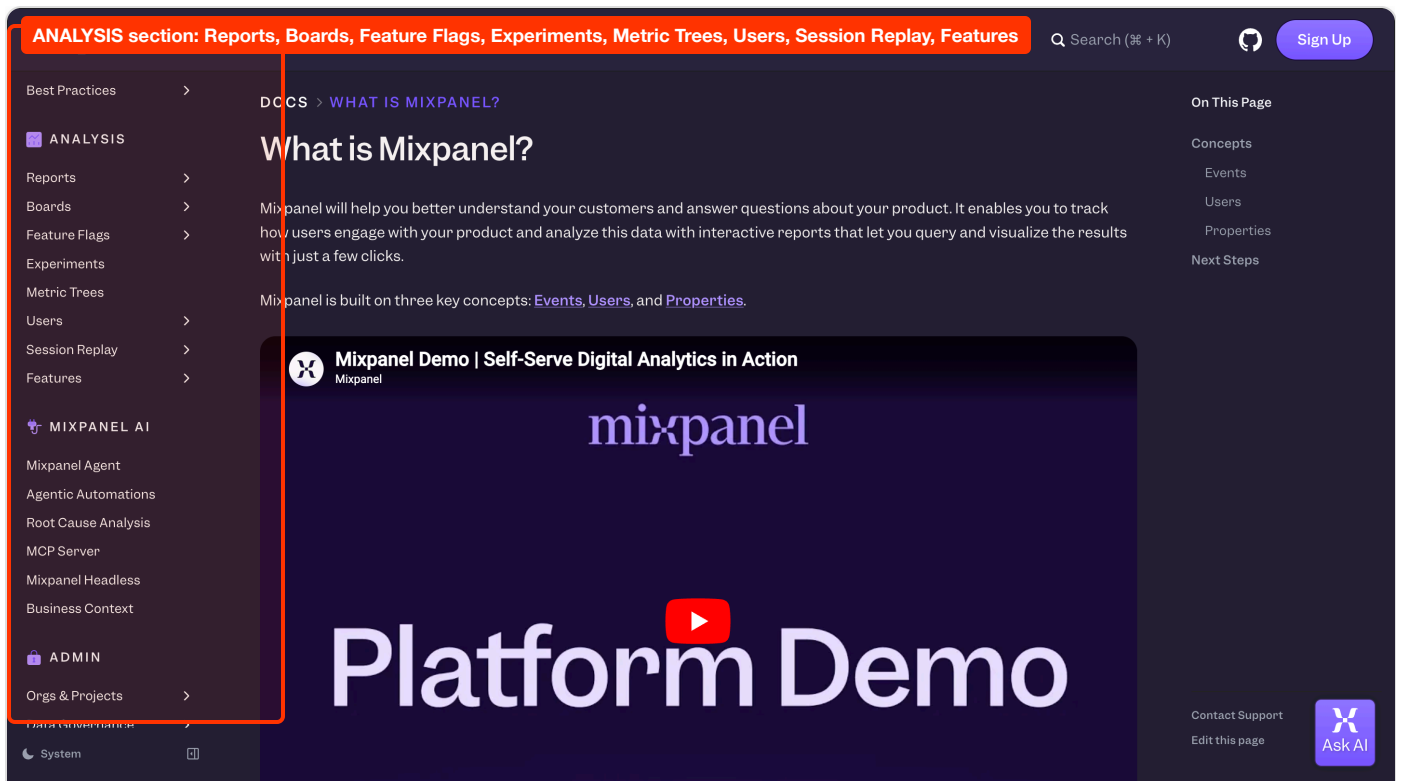
So when Nikhil references "DICs/dictionary" in the context of Mixpanel, he means Lexicon — the governance/metadata layer that describes what each event and property means. This is analogous to what we might build in the Observe reference section for our Trace Filter Syntax and Dashboard Metric Definitions pages.

The screenshot shows the Mixpanel Lexicon page. A red-bordered box highlights the title "Lexicon: Describe your events and data using a dictionary" and the subtitle "Overview #". Below the title, there is a table of events. A red play button icon is overlaid on the table. The table has columns for Event Name, Display Name, Description, Status, Volume, and Queries. The events listed are: Complete Purchase, Add To Cart, Item Detail Page, Search, and Delivery Method Added.

Event Name	Display Name	Description	Status	Volume	Queries
Complete Purchase		Triggered when a user completes their purchase in the checkout flow.	Visible	199.1K	12,99K
Add To Cart		Triggered when a user adds an item to their cart.	Visible	225.1K	6,43K
Item Detail Page		Triggered when viewing the details page of an item.	Visible	608.9K	5,75K
Search		Triggered when a search for an item is conducted from the search bar.	Visible	231.8K	5,61K
Delivery Method Added		Triggered when a user adds a delivery method.	Visible	6,664	1,66K

15 — Mixpanel Lexicon: found it — the page title literally says "using a dictionary." This is what Nikhil meant by "DICs." Lexicon is how Mixpanel makes sure every team member understands what each event means. Our Dashboard Metric Definitions page is analogous, but it's nowhere near this prominent or this clearly named.

Found it. This is not a subtle thing — the page title literally says "using a dictionary." Lexicon is how Mixpanel ensures everyone on the team understands what events mean. We have something analogous in our Dashboard Metric Definitions reference page, but it's not this prominent or as well-named.



11 — Mixpanel sidebar (ANALYSIS section): the sidebar scrolled to show the ANALYSIS section header followed by: Reports, Boards, Feature Flags, Experiments, Metric Trees, Users, Session Replay, Features. Below it sits MIXPANEL AI (Mixpanel Agent, Agentic Automations, Root Cause Analysis, MCP Server, Mixpanel Headless, Business Context) and the start of ADMIN (Orgs & Projects, Data Governance).

The ANALYSIS section is the cleanest part of Mixpanel's sidebar — most items are 1–2 words. MIXPANEL AI sits directly beneath it as a distinct section, which is smart positioning: once you've analyzed something, AI-powered follow-ups are the natural next step. The ALL-CAPS landmarks make the section break visible even when scanning fast.

Step 2: Choose your measurement — the core of the Insights flow

Step 2: Choose your Measurement

Once you've selected your behavior, you can choose how to measure that behavior. A behavior combined with a measurement forms a Metric.

Depending on the type of behavior selected, Insights will default to a different measurement. If you're measuring a Simple Behavior (events), Insights will default to measuring the number of unique users who have done that event.

Date	May 4	May 5	May 6
Uniques	266	327	329

The right sidebar shows the configuration for the metric: 'Sign Up Completed' measured by '# Unique Users'. A dropdown menu is open, showing options like 'Total Events', 'Total Sessions', 'Frequency per User', 'Aggregate Property', and 'Aggregate Property per User'. The 'Unique Users' option is highlighted.

💬 This is really well-done. The step structure turns what could be a dumped feature list into a decision tree. You're never lost because you know which step you're on. Step 2 in particular — choosing your measurement (total, unique users, sum, min, max, etc.) — is where most users get confused, and Mixpanel doesn't skip it. We should do the same for our Dashboards.

The screenshot displays the Mixpanel documentation interface. On the left, a sidebar lists navigation options under 'What to Track' and 'ANALYSIS'. The main content area is titled 'Step 3: Choose Filters' and includes a line chart showing data trends from May 24 to May 30. To the right of the chart is a configuration panel for 'Uniques of Sign Up Completed', showing a filter for 'Operating System' set to 'Is iOS'. A red box highlights the title 'Step 3: Choose Filters — Operating System Is iOS — filter applied in UI'. On the right side, a 'On This Page' table of contents lists various sections like 'Overview', 'Use Cases', 'Quick Start', and 'Basic Features'.

13 — Mixpanel Insights Step 3 (Choose Filters): the Insights guide at Step 3: "Filters exclude unwanted data. In this case, we only care about events performed on the iOS platform. Therefore, add an 'Operating System' filter, where OS equals 'iOS'." The embedded UI screenshot shows the filter applied: Operating System → Is → iOS. The right-rail TOC confirms this is Step 3 of 6, with Step 4: Choose Breakdowns next.

This is the pattern that makes Mixpanel's docs so usable — every step shows you exactly what to do in the UI, with a screenshot that matches. You're never guessing where to click. Each step is anchor-linked so you can share "here's exactly where you are" with a teammate. Step 3 of the filter flow is the one most users get stuck on, and Mixpanel doesn't skip it.

What Mixpanel Does Best

Step-numbered feature flows. Their Insights and Funnels docs use numbered steps that map directly to UI actions — you can follow the doc while having the product open. The Boards architecture is also well-framed: Board contains pinned Insights reports, each shareable via a URL. The Lexicon data dictionary concept is prominently surfaced under Data Governance.

Moves We Steal from Mixpanel

- Step-by-step numbered flows for feature docs (Dashboards → Add Widget → Configure → Pin to view)
- ALL-CAPS section dividers as sidebar landmarks
- The "Boards" mental model: collection surface vs. monitoring view
- Anchor-linked steps that can be shared as URLs (each step = a section = a link)
- Prominent data dictionary surface under Governance (our Dashboard Metric Definitions → upgrade it)

PostHog

Most intuitive overall flow · AI Observability product · Filters (assessed: nested under Trends, real depth)

RESOLVED: PostHog Observability Product Name

PostHog's LLM/AI observability product is called "**AI Observability**". The URL is `posthog.com/docs/ai-observability`. We did not find a legacy `/llm-observability` slug in the captured navigation or screenshots — the product page was captured directly at `/docs/ai-observability`. The product is NOT called "Observe" — that label does not appear anywhere in their nav. Their heading on the page is: "*AI Observability — Capture traces, generations, and spans from your AI and LLM products.*"

Relevant Subtree: AI Observability IA

AI OBSERVABILITY (product selector)

Overview

Getting started (subsection)

Start here

Installation (expandable)

Privacy mode

Concepts (subsection)

Basics · Generations · Traces · Spans · Sessions · Errors · Tools · Embeddings

Evaluations (expandable) · Prompt management (expandable)

Guides (subsection)

Calculating LLM costs · Analyze LLM performance · Tracking custom properties

Link Session Replay · Link Error Tracking · Collect user feedback

PostHog AI (subsection)

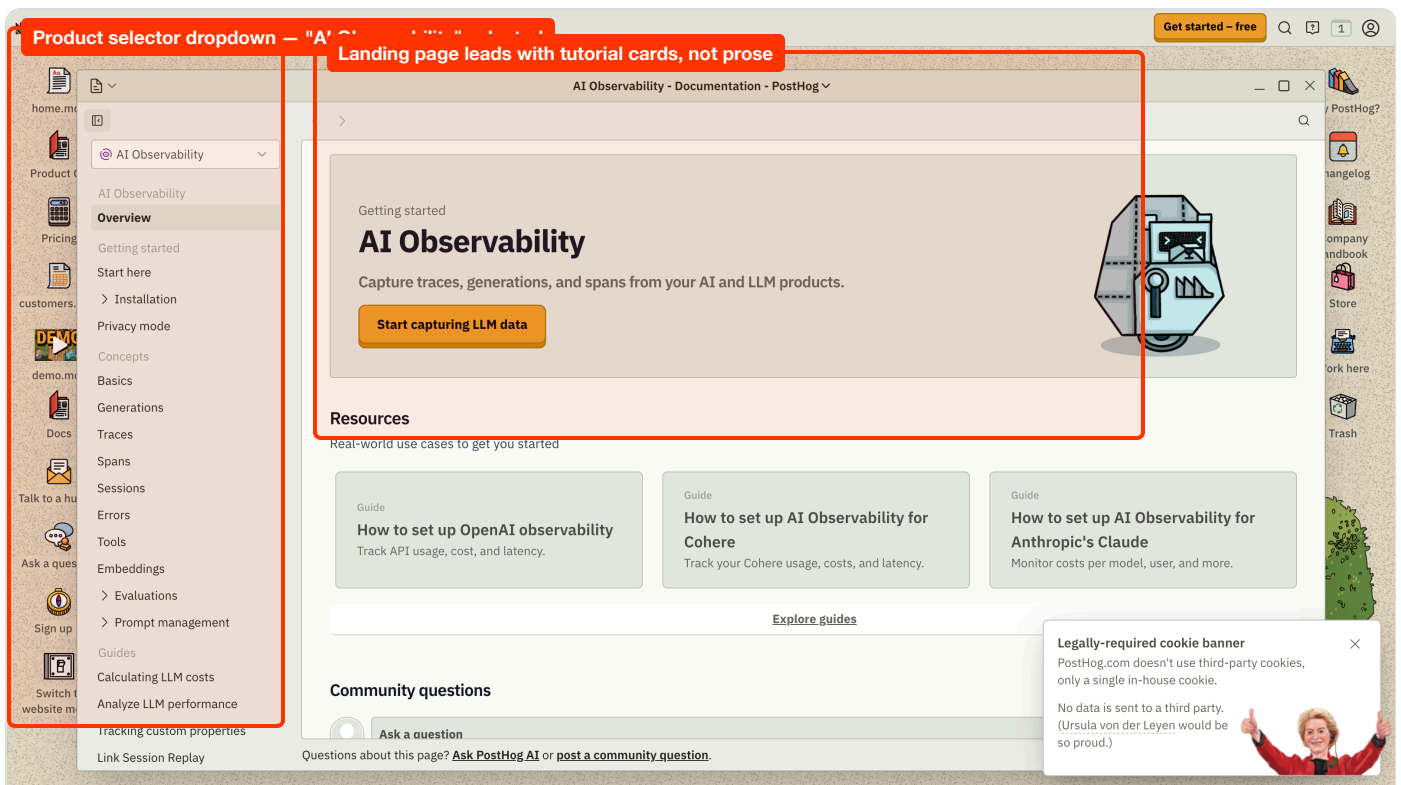
Analyze traces with PostHog AI · Query traces with MCP · Playground

Trace summarization · Clusters · Sentiment

Resources (subsection)

Demos & examples · Data retention · Troubleshooting · Changelog · More tutorials

Third-party integrations (expandable)



21 — PostHog AI Observability overview: nice framing — they open with "Start capturing LLM data," an action not a definition. The resources are tutorial cards linking straight to real integrations (OpenAI, Cohere, Anthropic). This is "show me how" instead of "let me tell you what this is." The product name here is unambiguously "AI Observability."

Nice framing. They open with "Start capturing LLM data" — an action, not a definition. The resources are tutorial cards that take you straight to real integrations. This is closer to "show me how" than "let me tell you what this is." The product selector dropdown in the left rail switches between PostHog's products cleanly.

Product OS Pricing Docs Community Company More

Get started - free

Traces page: timeline, tool calls, sentiment, search — all in one page

Traces

Traces are a collection of **generations** and **spans** that capture a full interaction between a user and an LLM. The **traces tab** lists them along with the properties autocaptured by PostHog like the person, total cost, total latency, and more.

Sessions vs Traces

- Trace** (`$ai_trace_id`): Groups related generations and spans together. Required for all AI Observability events.
- Session** (`$ai_session_id`): Optional property that groups multiple traces together based on your chosen grouping strategy.

See the [Sessions](#) documentation for more details on how to use `$ai_session_id`.

Trace timeline

Clicking on a trace opens a timeline of the interaction with all the generation and span events. The trace timeline enables you to see the entire conversation, profiling details, and the individual generations and spans.

Jump to:

- Sessions vs Traces
- Trace timeline
- Conversation display options
- Tool calls
- Sentiment classification
- Search traces with PostHog AI
- AI event hierarchy
- Event properties

Traces

25.9 s • 8216 • 127 (2.8345) / 80.0125

LangGraph

25.9 s • 8216 • 127 (2.8345) / 80.0125

SPAN ..._span_1

6.82 s • 0 • 0 (2.0) / 50.00

Tree

Search trace...

LangGraph

25.9 s • 8216 • 127 (2.8345) / 80.0125

SPAN ..._span_1

6.82 s • 0 • 0 (2.0) / 50.00

Conversation

Rate

Prompt

— E

Copy Trace

Display Options

25.9 s of latency • 8216 prompt tokens • 127 completion tokens (2.8345) • 80.0125

Legally-required cookie banner

PostHog.com doesn't use third-party cookies, only a single in-house cookie.

No data is sent to a third party. (Ursula von der Leyen would be so proud.)

Questions about this page? [Ask PostHog AI](#) or [post a community question](#).

25 — PostHog Traces depth: this is good depth — the Traces page isn't just "traces are collections of spans." It tells you what the timeline shows, how tool calls are displayed, that the Traces page documents sentiment classification (per its own TOC entry), and that PostHog AI can search traces by date/cost/model. Every concept has a concrete behavioral description, and you can see all of it on one page.

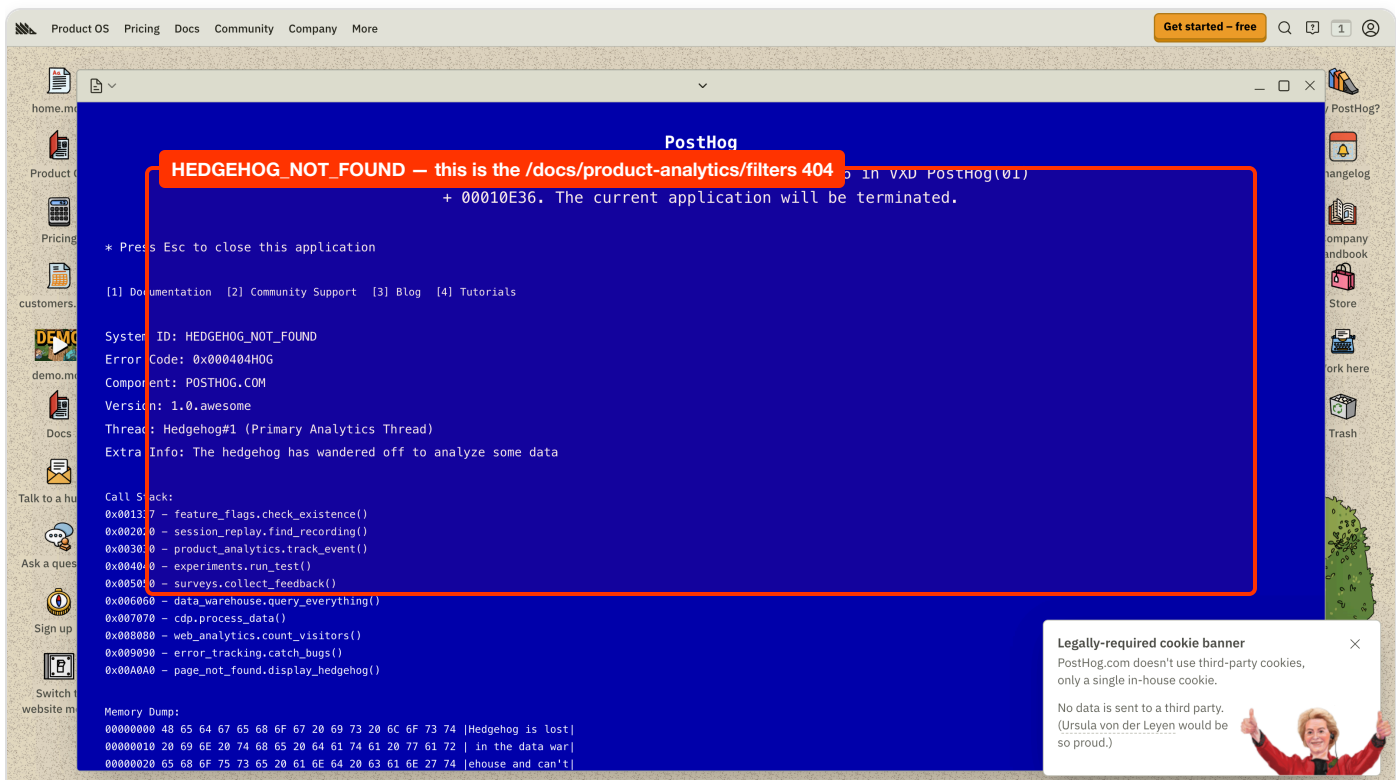
This is good depth. The Traces page isn't just "traces are collections of spans." It tells you what the trace timeline shows, how tool calls are displayed, that sentiment classification is documented as a named concept in the Traces TOC, and that PostHog AI can search across traces by date/cost/model. Every concept has a concrete behavioral description. That's the level we should write at.

Filters Verdict: PostHog vs Langfuse

Correction: The `/docs/product-analytics/filters` 404 was a wrong URL guess, not proof of absence

The original verdict leaned on `22-posthog-filters.png` showing PostHog's joke 404 page (HEDGEHOG_NOT_FOUND) at `/docs/product-analytics/filters`. That slug doesn't exist — but PostHog *does* have a real, substantive Filters page nested under Trends at `/docs/product-analytics/trends/filters`. It appears in the sidebar as **Trends** → **Filters**. A 404 on a guessed URL is not evidence of absence.

The actual page covers: two filter types (Inline filters vs Filter groups), AND/OR operator rules, a 3-element filter configuration model (property + operation + value), a full operator reference table (equals, contains, regex, greater/less than, is set, etc.), filtering internal and test users, and first-occurrence matching filters for user. This is real depth.



22 — PostHog filters 404 (wrong URL): this is the HEDGEHOG_NOT_FOUND page that was originally cited as evidence PostHog lacks filter docs. It's PostHog's joke 404. The slug `/docs/product-analytics/filters` simply doesn't exist — the real page lives at `/docs/product-analytics/trends/filters`. Keeping this screenshot only as honest evidence that the guessed URL 404s, not as proof of absence.

OK, so this was my mistake. A 404 on a guessed URL is not evidence that the content doesn't exist — it just means I guessed the wrong slug. The real filters page is nested under Trends, not at the top level of product-analytics. Important distinction. Let me go find the actual page.

Sidebar: Trends → Filters — Real Filters page: inline filters, filter groups, AND/OR, Right-rail TOC: Configuring filters, Types, Filtering internal users

Filters

Trends support two **types of filters**:

1. **Inline filters** – only apply to a specific series in your insight.
2. **Filter groups** – global filters that apply to the whole insight.

Inline filters only support **AND** operators – i.e. all filter conditions must be met for an event or user to be included.

Filter groups support **AND** and **OR** operators – i.e. events will be included if one or both of the filter conditions are met.

Configuring filters

Filters are comprised of three elements, a property, an operation, and a value.

Note: You can also fully customize your filter by writing them as a [SQL expression](#). See our tutorial on [using SQL for advanced time and date filters](#) for an example.

1. The filter property

Example: (Current URL, Browser, etc)

Questions about this page? [Ask PostHog AI](#) or [post a community question](#).

Jump to:

- Configuring filters
 1. The filter property
 2. The filter operation
 3. The comparison value
- Types of filters
 - Inline filters
 - Filter groups
- Filtering internal and test users
- Filtering by first event occurrence
 1. First-ever occurrence for user
 - First occurrence matching filters for user

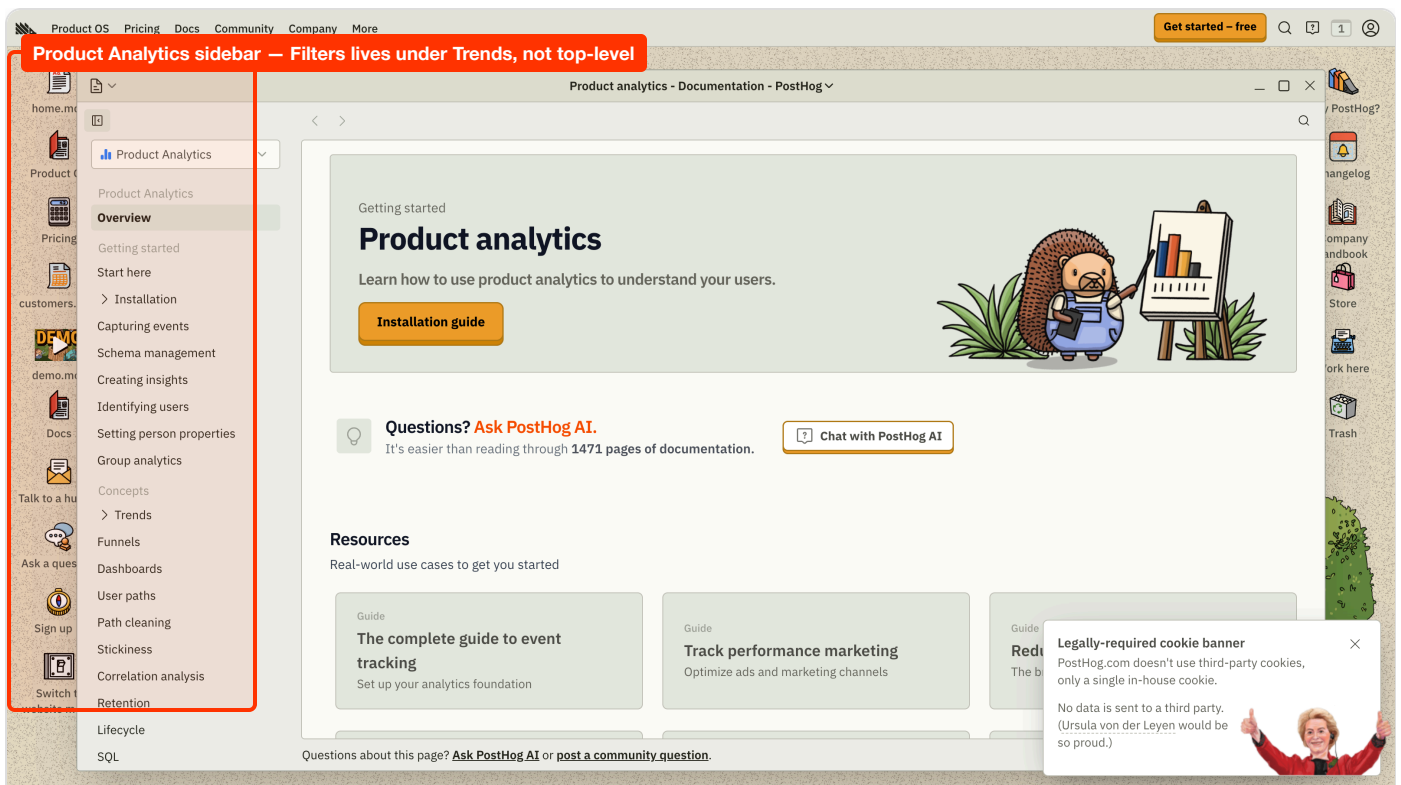
Legally-required cookie banner

PostHog.com doesn't use third-party cookies, only a single in-house cookie.

No data is sent to a third party. (Ursula von der Leyen would be so proud.)

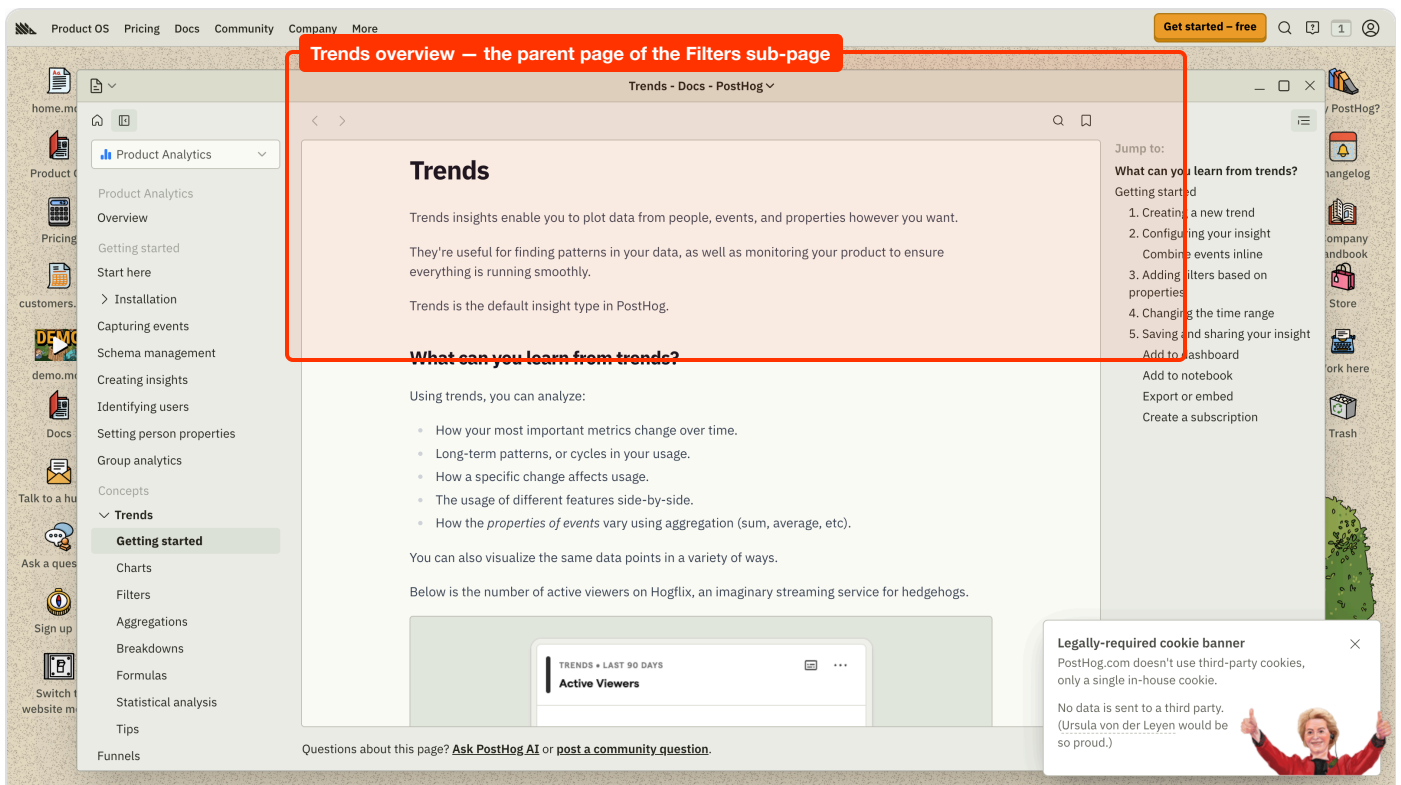
27 — PostHog real Filters page (/docs/product-analytics/trends/filters): found it — a genuine, substantive filters page. It explains two filter types (Inline filters apply per-series; Filter groups apply across all series), AND vs OR operator semantics, and a full operator reference table. The right-rail TOC shows real section depth: Configuring filters → The filter property → The filter operation → The comparison value → Types of filters → Filtering internal and test users → First-occurrence matching filters for user.

OK this is real depth. Property, operation, value — that's the 3-part filter model, and PostHog explains each part. The operator table lists equals, contains, regex, greater-than, less-than, is-set, is-not-set — all of them. The distinction between Inline filters (per-series, AND only) and Filter groups (all series, AND or OR) is a genuinely non-obvious concept and PostHog documents it clearly. This upgrades their score.



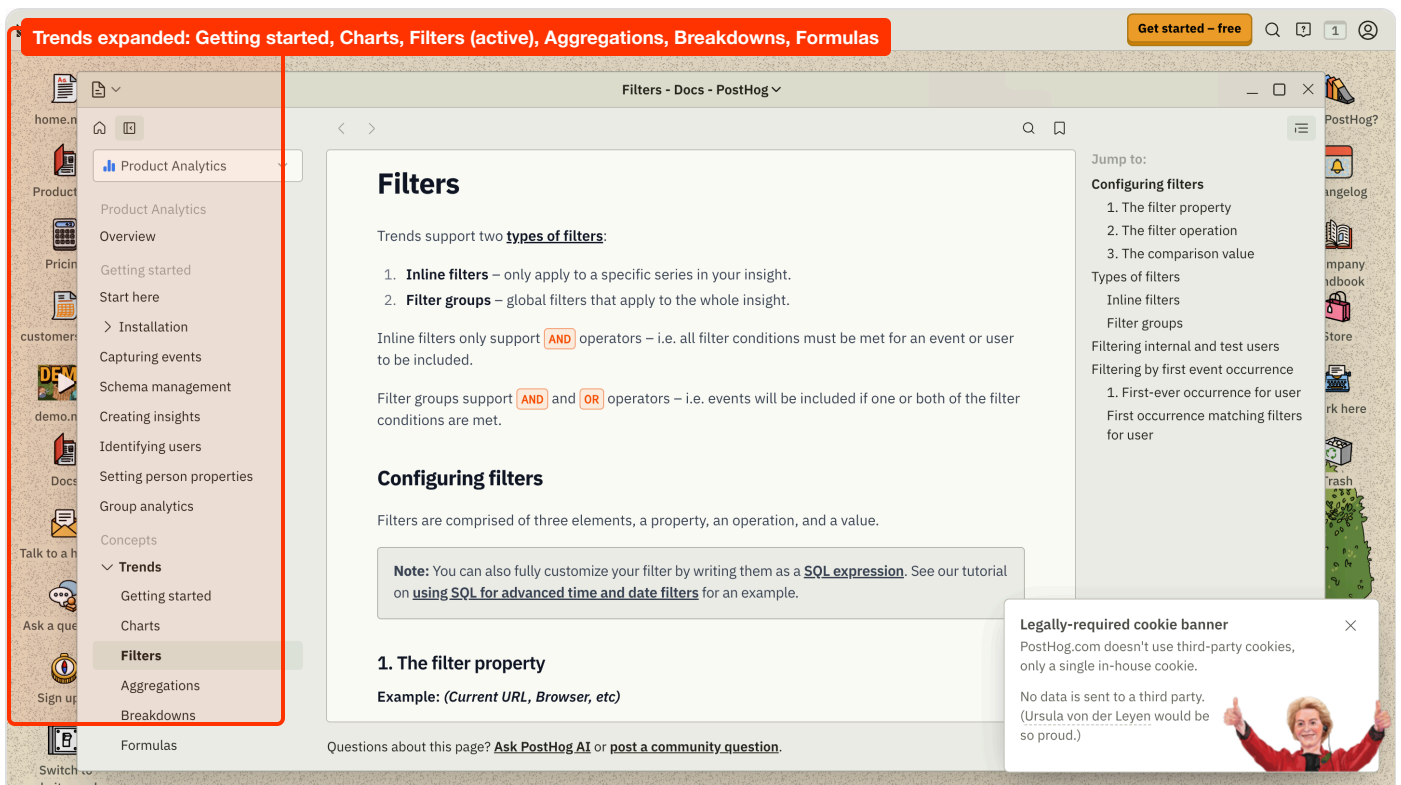
23 — PostHog Product Analytics sidebar: the visible sidebar shows: Overview, Getting started, and Concepts (with items including Trends, Funnels, Dashboards and others visible in-frame). No top-level "Filters" entry is present in the visible portion, which is why the original guess of `/docs/product-analytics/filters` 404'd. Filter documentation is nested under Trends → Filters. Guides, PostHog AI, and Resources sections are below the fold in this frame and not shown.

So filters aren't a top-level item — they're a sub-page of Trends. That's actually a reasonable IA decision: filters are primarily a Trends-level concept, and if you're looking at Funnels or Retention the filter behavior is documented inline there. It's not a perfect model (you'd want a shared filters reference eventually) but it's not "no filter docs" either.



24 — PostHog Trends page: this is the parent of the Filters sub-page. Trends itself is a full concept — charts, aggregations, breakdowns, formulas — and Filters is one of its sub-topics. The nesting makes sense: you understand Trends first, then you learn to filter within Trends. But it does mean filter docs aren't discoverable from the top-level sidebar without expanding Trends first.

The Trends section is dense — Charts, Filters, Aggregations, Breakdowns, Formulas, Statistical analysis, Tips — all as sub-pages. That's real depth for a single feature. The downside is discoverability: if I land on the Product Analytics overview and scan the sidebar for "Filters" I won't see it without expanding Trends. That's a UX tradeoff PostHog has made.



26 — PostHog sidebar (Trends sub-tree, Filters active): the Product Analytics sidebar with the Trends sub-tree expanded showing: Getting started, Charts, Filters (currently active), Aggregations, Breakdowns, Formulas. The main area is the Filters page. The Filters page itself covers inline filters vs. filter groups, AND/OR operators, and the three-element configuration (property, operation, comparison value).

Filters is the fourth item inside Trends — right after Charts, which is exactly where you'd look for it. The sub-tree expansion shows PostHog treats each concept (Charts, Filters, Aggregations, Breakdowns) as a distinct page, not a collapsed section. That's the per-concept-page discipline we should bring to our own Observe features.

Revised Filters Verdict: Langfuse still wins, but PostHog is not absent — it's differently structured

Langfuse model: every filterable attribute (Sessions, Tags, Environments, Log Levels, Metadata, Full-Text Search) gets its own dedicated page in the Features section. The filter docs live with the feature docs, not in a separate filters section.

PostHog model: a dedicated Filters page nested under Trends (</docs/product-analytics/trends/filters>) explains the filter configuration model (property + operation + value), operator types, AND/OR semantics, inline vs group filters. Real depth on the mechanics of filtering. But it's specific to the Trends report type — Funnels, Retention, etc. each document their own filter behavior inline.

Verdict: Langfuse still wins on the per-attribute model. PostHog documents *how filters work mechanically* well. Langfuse documents *what each filterable attribute means and how to set it up* — one page per attribute, from SDK setup to UI usage to edge cases. For our Observe docs revamp, the Langfuse per-attribute-page model is the right target. PostHog's operator-reference table is also worth borrowing for our Trace Filter Syntax page.

What PostHog Does Best

Product-switch sidebar that keeps products crisp and isolated. Lifecycle-ordered IA within each product: Overview → Getting Started → Concepts → Guides → AI-powered → Resources. The flow feels like a deliberate tutorial path, not a feature dump. Action-first copy throughout ("Start here," "Start capturing LLM data").

Moves We Steal from PostHog

- The Overview → Getting started → Concepts → Guides → Resources IA sequence for our Observe product
- Action-first sidebar labels: "Start here" (not "Introduction"), "Start capturing" (not "Setup")
- Tutorial card grids on the product overview page linking to real integration guides
- Subsection grouping within a product sidebar (not just a flat list)
- Dedicated concept pages for every named entity: Traces, Spans, Sessions, Errors, Tools, Embeddings

Sentry

Install-first page ordering · right-side on-page heading tree

Relevant Subtree: Python Platform Docs

SENTRY FOR PYTHON

Quick Start ← FIRST PAGE IS THE INSTALL

FEATURES

Capturing Errors

Logs

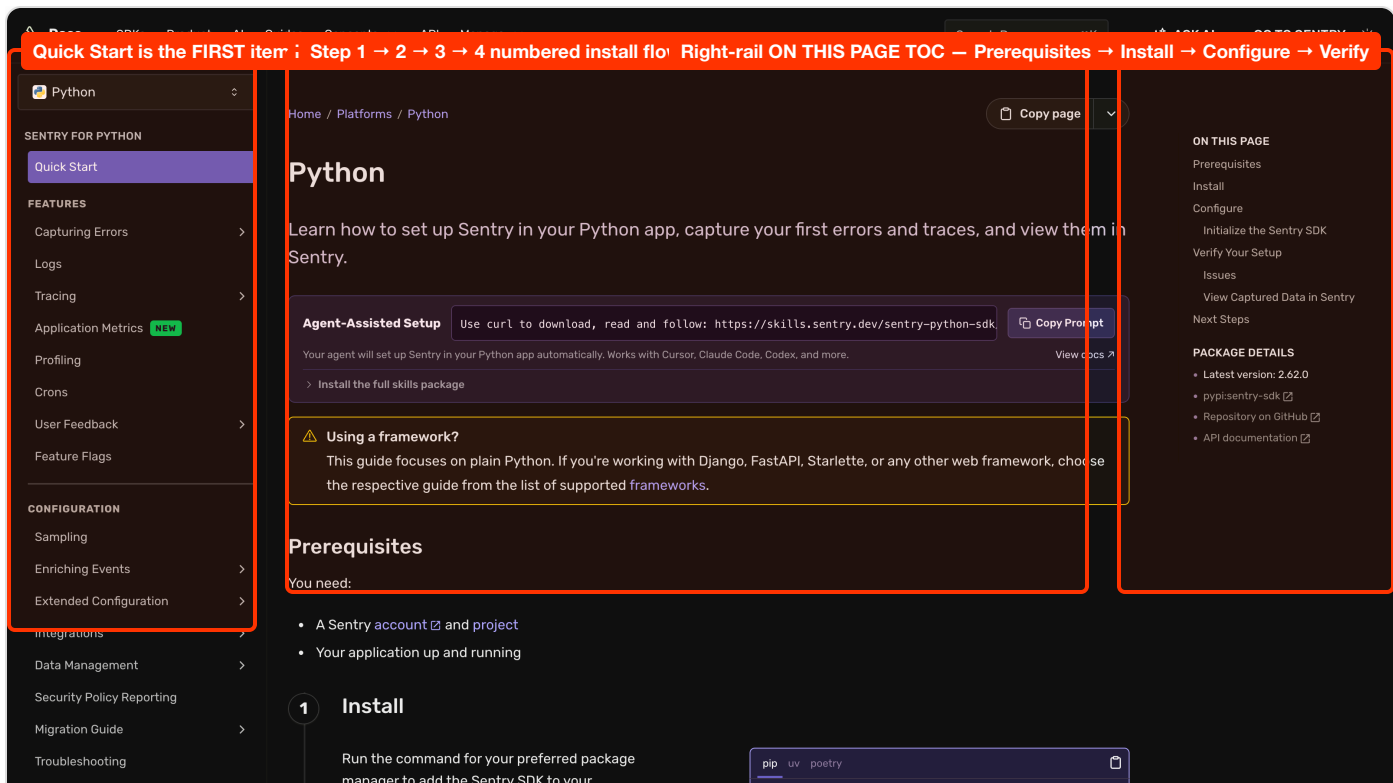
Tracing

Application Metrics NEW

Profiling · Crons · User Feedback · Feature Flags

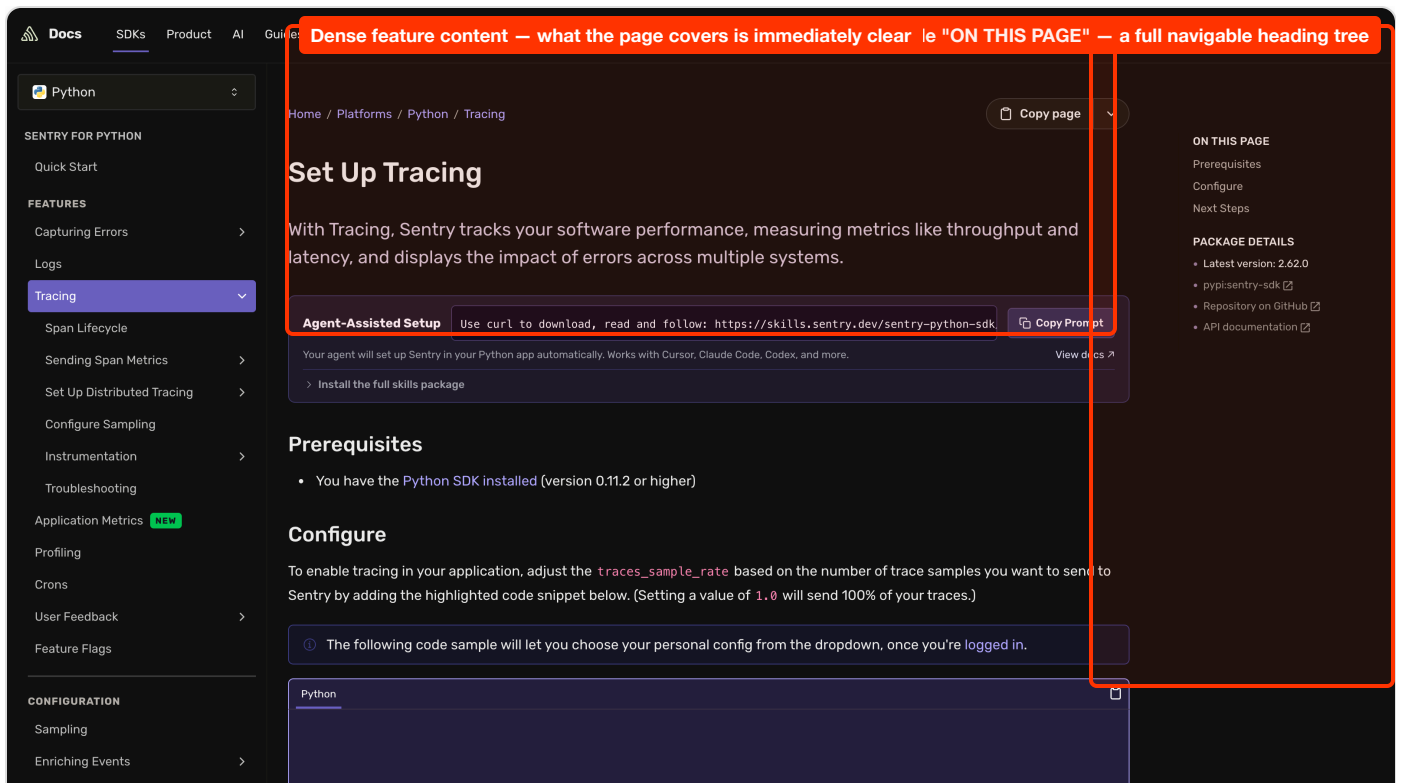
CONFIGURATION

Sampling · Enriching Events · Extended Configuration · Integrations · Data Management · Security
Policy Reporting · Migration Guide · Troubleshooting



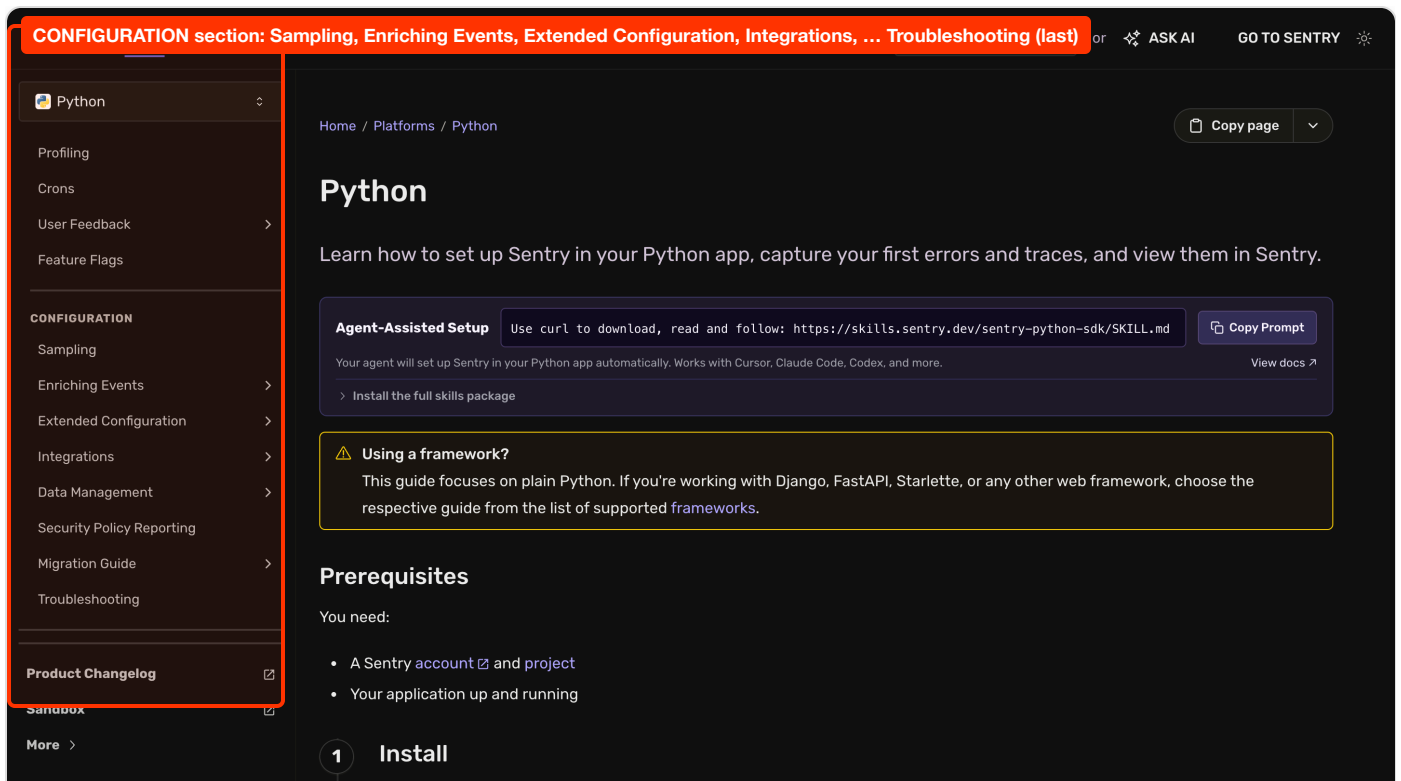
31 — Sentry Python Quick Start landing page: the Python platform landing page with Quick Start pre-selected in the sidebar — the install flow begins immediately. The right-side "ON THIS PAGE" tree lists: Prerequisites, Install, Configure, Initialize the Sentry SDK, Issues, View Captured Data in Sentry, Next Steps. You can see the entire install sequence before reading a single line of prose.

This is clean. The Python platform lands you directly on Quick Start — not "What is Sentry?", not Concepts. The right-side "ON THIS PAGE" tree (Prerequisites → Install → Configure → Initialize the Sentry SDK → Issues → View Captured Data in Sentry → Next Steps) is exactly the kind of navigation Braintrust also uses. When a page has 5+ sections, the right-rail saves you from scrolling blind.



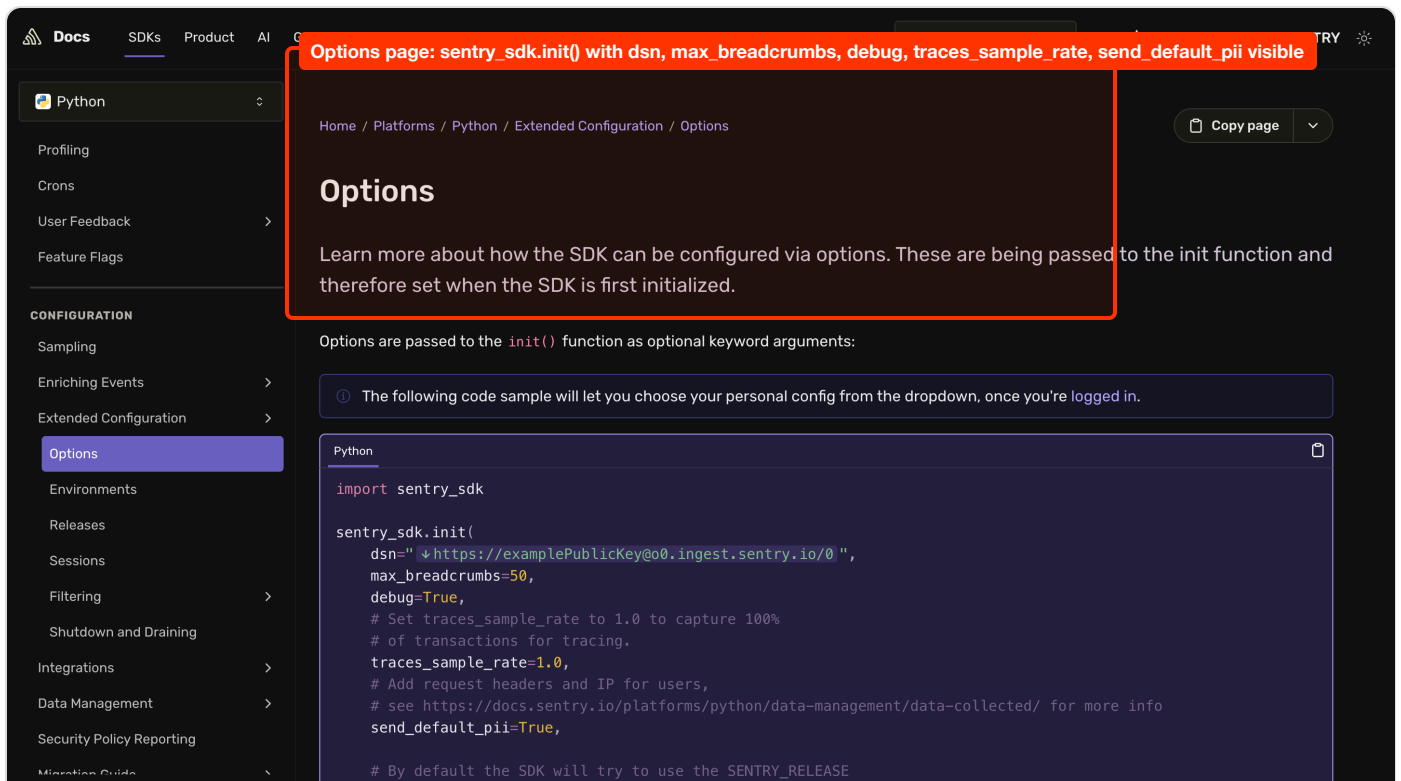
32 — Sentry on-page TOC (Tracing page): the right-rail heading tree turns a long page into a map — it persists as you scroll so you always know where you are. The PACKAGE DETAILS box at the bottom of the rail (SDK version, PyPI link, GitHub repo, API docs) is a nice bonus. Everything you'd need to reference is in one rail.

The right-side heading tree turns a long page into a map. The PACKAGE DETAILS box at the bottom of the right rail — showing the current SDK version, PyPI link, GitHub repo, API docs — is a nice touch. Every single thing you'd need to reference is in one rail. I want this for our pages.



33 — Sentry sidebar (Troubleshooting last in CONFIGURATION): the Sentry Python sidebar showing the CONFIGURATION section — Sampling, Enriching Events, Extended Configuration, Integrations, Data Management, Security Policy Reporting, Migration Guide, Troubleshooting — with Troubleshooting as the final item. Below CONFIGURATION sits Product Changelog, Sandbox, and More. The main area shows the Python platform home page.

Troubleshooting as the last item under CONFIGURATION is a deliberate choice — it signals the reader flow: set up first, debug when it breaks. Our sidebar currently puts Framework integrations after Troubleshooting, which inverts this. Moving Troubleshooting to last would cost zero content work and immediately align us with this pattern.



34 — Sentry configuration depth (Options page): the Python → Extended Configuration → Options page opens: "Learn more about how the SDK can be configured via options. These are being passed to the init function and therefore set when the SDK is first initialized." A Python code block shows `sentry_sdk.init()` with `dsn`, `max_breadcrumbs=50`, `debug=True`, `traces_sample_rate=1.0`, `send_default_pii=True` and inline comments explaining each parameter. The sidebar shows Options is highlighted inside CONFIGURATION.

The Options page leads with a real code block — not a table header — so you see the actual init call before you read anything. The inline comments explain each parameter in context: "Set traces_sample_rate to 1.0 to capture 100% of transactions for tracing." That's parameter documentation embedded in working code, not a separate table you have to cross-reference. We have no equivalent for our tracing init options.

What Sentry Does Best

Install as the entry point — you start coding immediately. The on-page right-rail heading tree is the best implementation of this pattern I saw across all six subjects. Parameter tables for every config option. Troubleshooting is last in the sidebar, always.

Moves We Steal from Sentry

- Right-side "ON THIS PAGE" heading tree on every content-rich page
- Quick Start / Getting Started as the very first sidebar item
- Parameter tables for every configuration option (with type, default, and description columns)
- Troubleshooting as the very last sidebar item — after everything else
- FEATURES and CONFIGURATION as ALL-CAPS section dividers

Langfuse

Filters-depth WINNER · per-feature-attribute pages · open source transparency

Relevant Subtree: Observability Product

OBSERVABILITY (product)

Overview

Get Started

Concepts

► Features (expandable — two sub-groups: Essential and Advanced)

Features → Essential

Sessions · User Tracking · Environments · Tags · Metadata

Trace IDs & Distributed Tracing

Features → Advanced

Comments · Corrections · User Feedback · Log Levels

Agent Graphs · Full-Text Search · Masking · MCP Tracing

Multi-Modality · Observation Types · Event queuing/batching

Releases & Versioning · Sampling · Token & Cost Tracking · Trace URLs · Web Callouts

► SDKs (expandable)

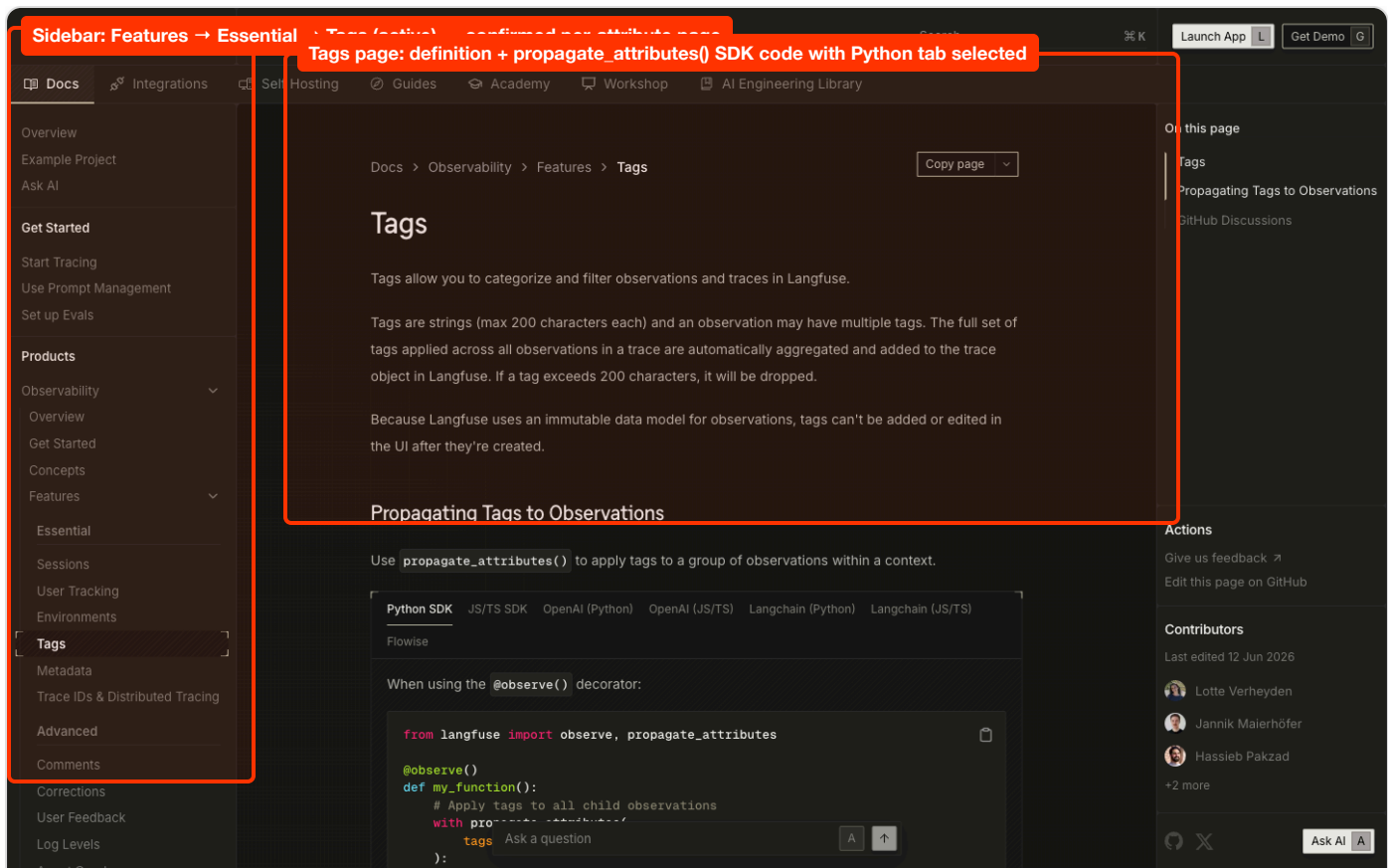
Integrations ↗

Troubleshooting & FAQ

VERDICT: Langfuse Wins the Filters Deep-Dive

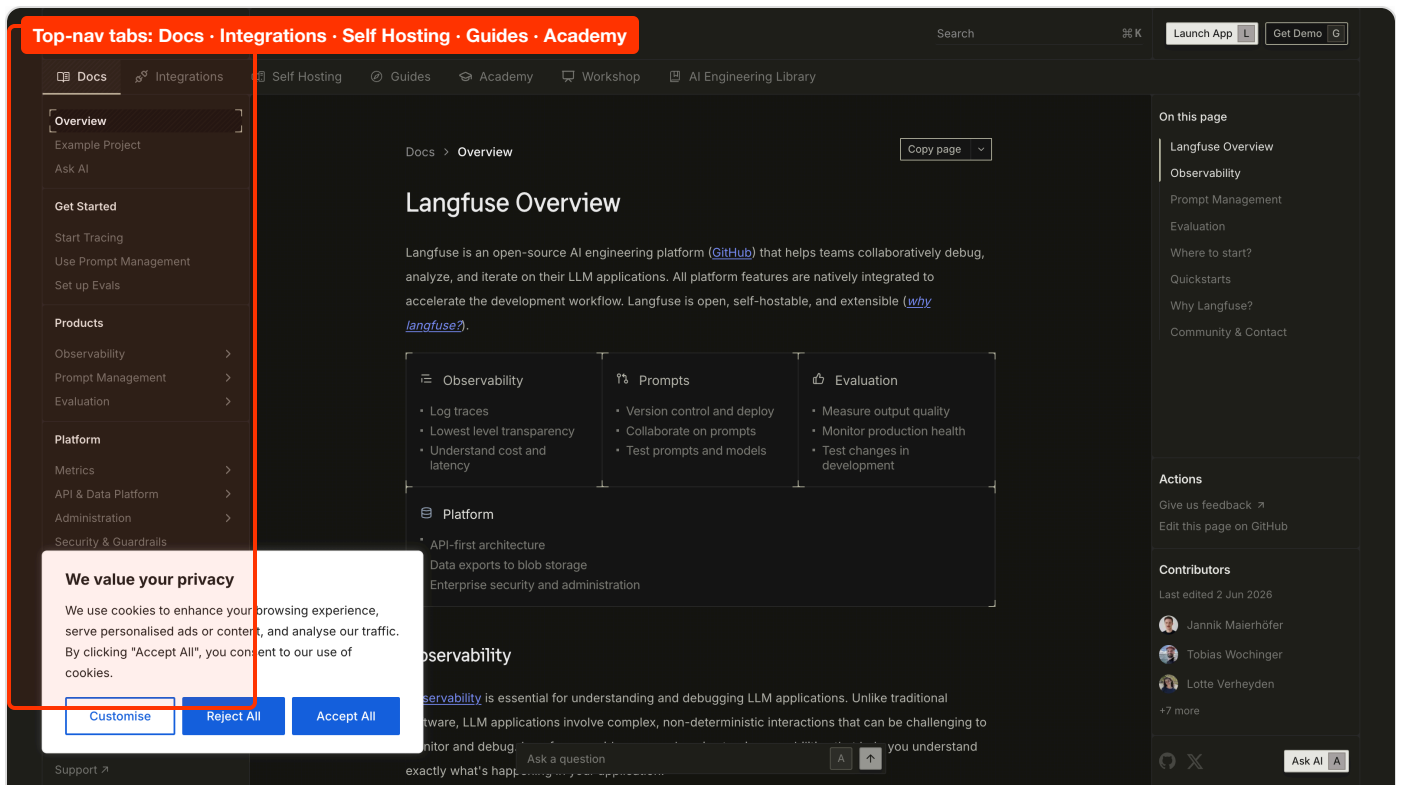
Langfuse gives every filterable attribute its own dedicated page under Features → Essential / Advanced. Confirmed in the live sidebar: Sessions, User Tracking, Environments, Tags, Metadata, Log Levels, Full-Text Search — each a full page. Fig 46 below shows the Tags page as a representative example: it defines the attribute, shows multi-SDK code for setting it, and documents propagation rules and edge cases (200-char limit, immutable after creation). PostHog documents *how filters work mechanically* (property + operation + value, AND/OR semantics) on a single Trends sub-page, but never documents what each filterable attribute *is* or how to set it up. Langfuse wins because it covers both.

The per-attribute-page filter model is what "every filter documented individually" looks like in practice. This is the move to steal for our Observe docs.



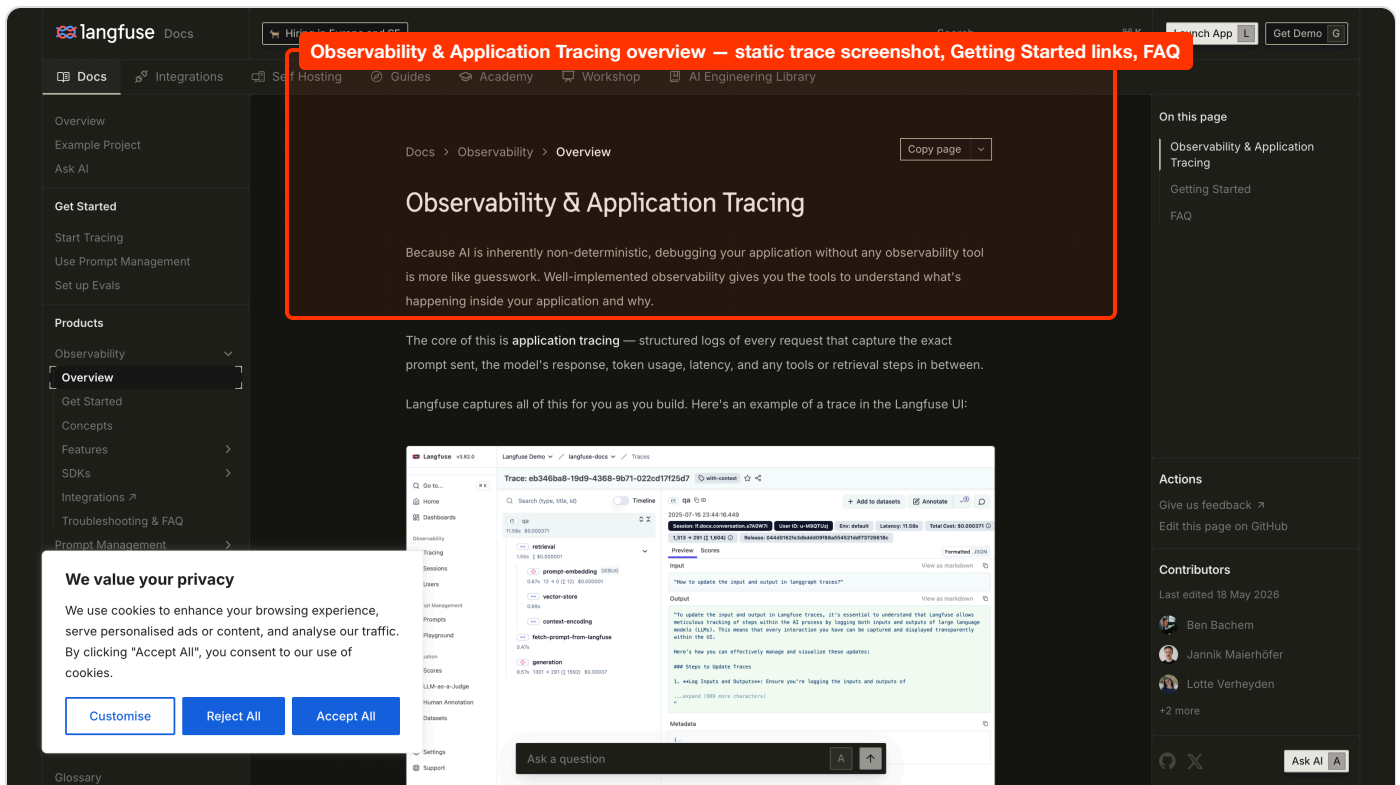
46 — Langfuse Tags per-attribute page (langfuse.com/docs/observability/features/tags): the Tags page under Features → Essential in the left sidebar. The page defines tags as strings (max 200 characters) that can be aggregated across observations into the trace object. It shows SDK code using `propagate_attributes(tags=["tag-1", "tag-2"])` with tabs for Python SDK, JS/TS SDK, OpenAI (Python), OpenAI (JS/TS), LangChain (Python), LangChain (JS/TS), and Flowise. The right-rail "On this page" TOC lists: Tags → Propagating Tags to Observations → GitHub Discussions. The sidebar confirms the Essential group also includes Sessions, User Tracking, Environments, Metadata — each its own page.

This is the concrete proof behind the Filters verdict. A tag isn't just a filter option — it's a first-class attribute with its own SDK method, propagation semantics, and constraints (immutable after creation, 200-char limit, automatically aggregated to trace level). One page per attribute means users can go directly to "Tags" from the sidebar and get everything they need. We have no equivalent. The move is to add Tags, Environments, Metadata as full pages under Features, each following: definition → SDK setup → UI filtering → edge cases.



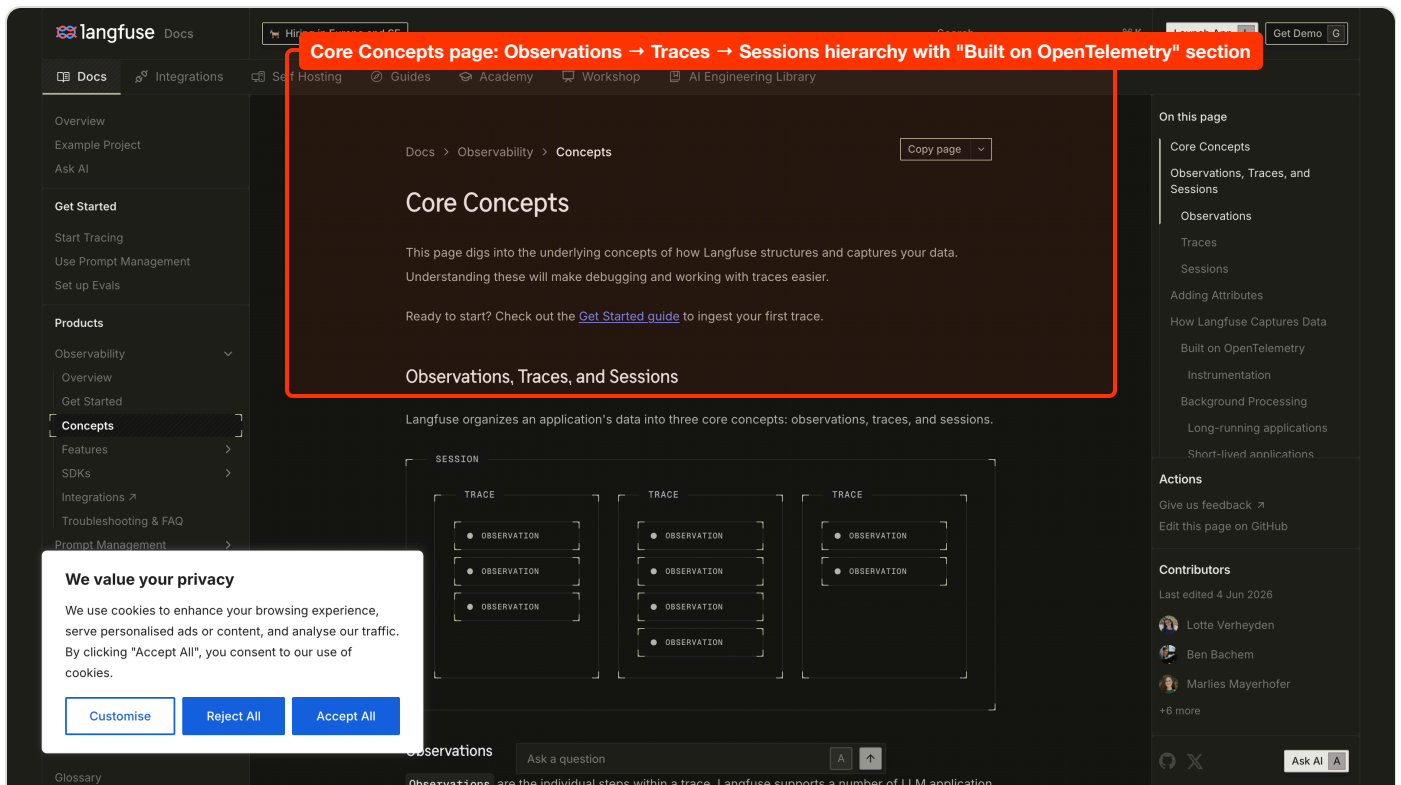
41 — Langfuse docs (top-level sidebar + top-nav): the top-nav tabs visible in this frame are: Docs, Integrations, Self Hosting, Guides, Academy, Workshop, and AI Engineering Library. The left sidebar shows the Observability product expanded with Overview, Get Started, Concepts, and Features (with the Features sub-tree open showing Essential and Advanced groups). This shot confirms the full top-nav scope — seven distinct tabs — and the sidebar's top-level Observability structure.

Seven top-nav tabs — Docs, Integrations, Self Hosting, Guides, Academy, Workshop, AI Engineering Library — signals that Langfuse treats documentation as a product surface, not just a reference dump. The sidebar product grouping (Observability expanded here, Prompt Management, Evaluation, Platform, More collapsed below) is clean progressive hierarchy. The Essential/Advanced split inside Features is visible in the sidebar tree transcription above, not from this viewport directly.



44 — Langfuse Observability overview: the page title is "Observability & Application Tracing." It opens with two paragraphs of conceptual framing about AI non-determinism and application tracing, then shows a static screenshot of a trace in the Langfuse UI. Below that is a Getting Started section with five action links (set up your first trace, understand concepts, group traces into sessions, add attributes, track costs) and an FAQ. No tabbed video demos are present on this page.

The overview page is conceptual framing plus a single static trace screenshot — clean but not interactive. The five Getting Started links are the real navigation layer: each one takes you to the right next step for your current situation. The FAQ at the bottom covers the "before I commit" questions a first-time user always asks. Simple structure that works.



42 — Langfuse data model / Concepts page: this is exactly the kind of unified Concepts page we're missing. Our current section has "What are Traces?" and "What are Spans?" as separate pages but no "here's how they all relate" page. Langfuse shows the hierarchy clearly: Observation is the base unit, Traces are collections, Sessions group multi-turn conversations. And it anchors everything to OpenTelemetry.

This is exactly the kind of Concepts page we're missing. Our current Concepts section has "What are Traces?" and "What are Spans?" as separate pages but no unified "here's how they relate" page. Langfuse's Core Concepts page shows the hierarchy: Observation is the base unit, Traces are collections, Sessions group multi-turn. Clean.

Token & Cost Tracking — "Ingest usage and/or cost" section with Python SDK code block

Via the [Metrics API](#), you can retrieve aggregated usage and cost metrics from Langfuse for downstream use in analytics, billing, and rate-limiting. The API allows you to filter by application type, user, or tags.

Ingest usage and/or cost

If available in the LLM response, ingesting usage and/or cost is the most accurate and robust way to track usage in Langfuse.

Many of the Langfuse integrations automatically capture usage details and cost details data from the LLM response. If this does not work as expected, please create an [issue](#) on GitHub.

Python SDK

When using the `@observe()` decorator:

```
from langfuse import observe, get_client
import anthropic

langfuse = get_client()
anthropic_client = anthropic.Anthropic()

@observe(as_type="generation")
def anthropic_completion(**kwargs):
    # optional, extract some fields from kwargs
    kwargs_clone = kwargs.copy()
    input = kwargs_clone.pop('messages', None)
    model = kwargs_clone.pop('model', None)
    langfuse.update_current_generation(
        input=input,
        model=model,
        metadata=...
```

On this page

- Model Usage & Cost Tracking
- Ingest usage and/or cost**
- Compatibility with OpenAI
- Infer usage and/or cost
- Usage
- Cost
- Pricing Tiers
- How tier matching works
- Custom model definitions
- Cost inference for reasoning models
- Troubleshooting
- GitHub Discussions

Actions

- Give us feedback ↗
- Edit this page on GitHub

Contributors

Last edited 18 May 2026

- Ben Bachem
- Max Deichmann
- Lotte Verheyden
- +4 more

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

[Customise](#) [Reject All](#) [Accept All](#)

43 — Langfuse feature page depth (Token & Cost Tracking, SDK section): the Token & Cost Tracking page scrolled to the "Ingest usage and/or cost" section showing a Python SDK tab with working code: `from langfuse import observe, get_client, @observe(as_type="generation")` decorator, and `langfuse.update_current_generation(input=input, model=model, metadata=...)`. The right-rail TOC confirms the page's full scope: Model Usage & Cost Tracking → Ingest usage and/or cost → Compatibility with OpenAI → Infer usage and/or cost → Usage → Cost → Pricing Tiers → Custom model definitions → Cost inference for reasoning models → Troubleshooting.

The right-rail TOC alone tells you this is a complete reference: nine distinct sections from ingestion to reasoning-model edge cases. The SDK code is real and runnable — not pseudocode. That's the depth standard we need to hit on our own feature pages: complete enough that a user never has to leave the page to figure out how to use the feature.

Pricing Tiers section: How tier matching works — Name, Priority, Conditions, Prices per tier

Pricing Tiers

Some model providers charge different rates depending on the number of input tokens used. For example, Anthropic's Claude Sonnet 4.5 and Google's Gemini 2.5 Pro apply higher pricing when more than 200K input tokens are used.

Langfuse supports **pricing tiers** for models, enabling accurate cost calculation for these context-dependent pricing structures.

How tier matching works

Each model can have multiple pricing tiers, each with:

- **Name:** A descriptive name (e.g., "Standard", "Large Context")
- **Priority:** Evaluation order (0 is reserved for default tier)
- **Conditions:** Rules that determine when the tier applies
- **Prices:** Cost per usage type for this tier

When calculating cost, Langfuse evaluates tiers in priority order (excluding the default tier). The first tier whose conditions are satisfied is used. If no conditional tier matches, the default tier is applied.

Condition format:

```

usageDetailPattern: A regex pattern to match usage detail keys (e.g., input matches
input_tokens, input_cached_tokens, etc.)
operator: Comparison operator (gt, gte, lt, lte, eq, neq)
value: The threshold value to compare against
caseSensitive: Whether the pattern matching is case-sensitive (default: false)

```

For example, the "L" Ask a question > 200000, meaning it applies when the sum of all usage details matching the pattern "input" exceeds 200,000

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

[Customise](#) [Reject All](#) [Accept All](#)

On this page

- Model Usage & Cost Tracking
- Ingest usage and/or cost
- Compatibility with OpenAI
- Infer usage and/or cost
- Usage
- Cost
- Pricing Tiers**
 - How tier matching works**
 - Custom model definitions
 - Cost inference for reasoning models
 - Troubleshooting
 - GitHub Discussions

Actions

- Give us feedback ↗
- Edit this page on GitHub

Contributors

Last edited 18 May 2026

- Ben Bachem
- Max Deichmann
- Lotte Verheyden
- +4 more

API & Data Platform >

45 — Langfuse Token & Cost Tracking (Pricing Tiers section): the same Token & Cost Tracking page, scrolled to the "Pricing Tiers" section and its sub-section "How tier matching works." Content: "Each model can have multiple pricing tiers, each with: Name (a descriptive name, e.g. Standard, Large Context), Priority (evaluation order), Conditions (rules that determine when the tier applies), Prices (cost per usage type)." The condition format is documented inline: `usageDetailPattern`, `operator` (`gt`, `gte`, `lt`, `lte`, `eq`, `neq`), `value`, `caseSensitive`.

This is a genuinely different section from shot 43 — we're now in the Pricing Tiers mechanics, not the ingestion SDK code. Four named properties per tier, operator types documented inline, a worked example about Claude Sonnet 4.5 and Gemini 2.5 Pro pricing thresholds. This is the edge-case depth that makes a feature page a reference, not just an overview.

Moves We Steal from Langfuse

- Per-filterable-attribute pages: one page per Tag / Session / Environment / User / Metadata attribute
- Essential vs. Advanced grouping within the Features section
- Per-attribute feature pages: every filterable attribute (Sessions, Environments, Tags, Metadata, Log Levels, Full-Text Search) gets its own dedicated page — this is what the Filters win is actually built on
- Getting-Started action links on the product overview page (Langfuse's five links: "set up your first trace / understand concepts / group traces into sessions / add attributes / track costs")
- A unified "Core Concepts" page that defines how entities relate to each other
- Top-nav tabs for Docs, Guides, Academy (signals docs-as-a-product)
- Troubleshooting & FAQ as the last item in every product sidebar

FutureAGI Observe (Ours)

The baseline — honest assessment of where we are vs. where we need to be

Current Sidebar — Transcribed Tree (from `navigation.ts`)

OBSERVE (section)

Overview

► Concepts

Understanding Observability

What are Traces?

What are Spans?

Sessions and Users

What is OpenTelemetry?

What is traceAI?

► Features

Set Up Observability

LLM Tracing

Sessions

Users

Run Evals on Traces

Dashboards

Alerts & Monitors

Voice Observability

► Manual Tracing (14 sub-pages)

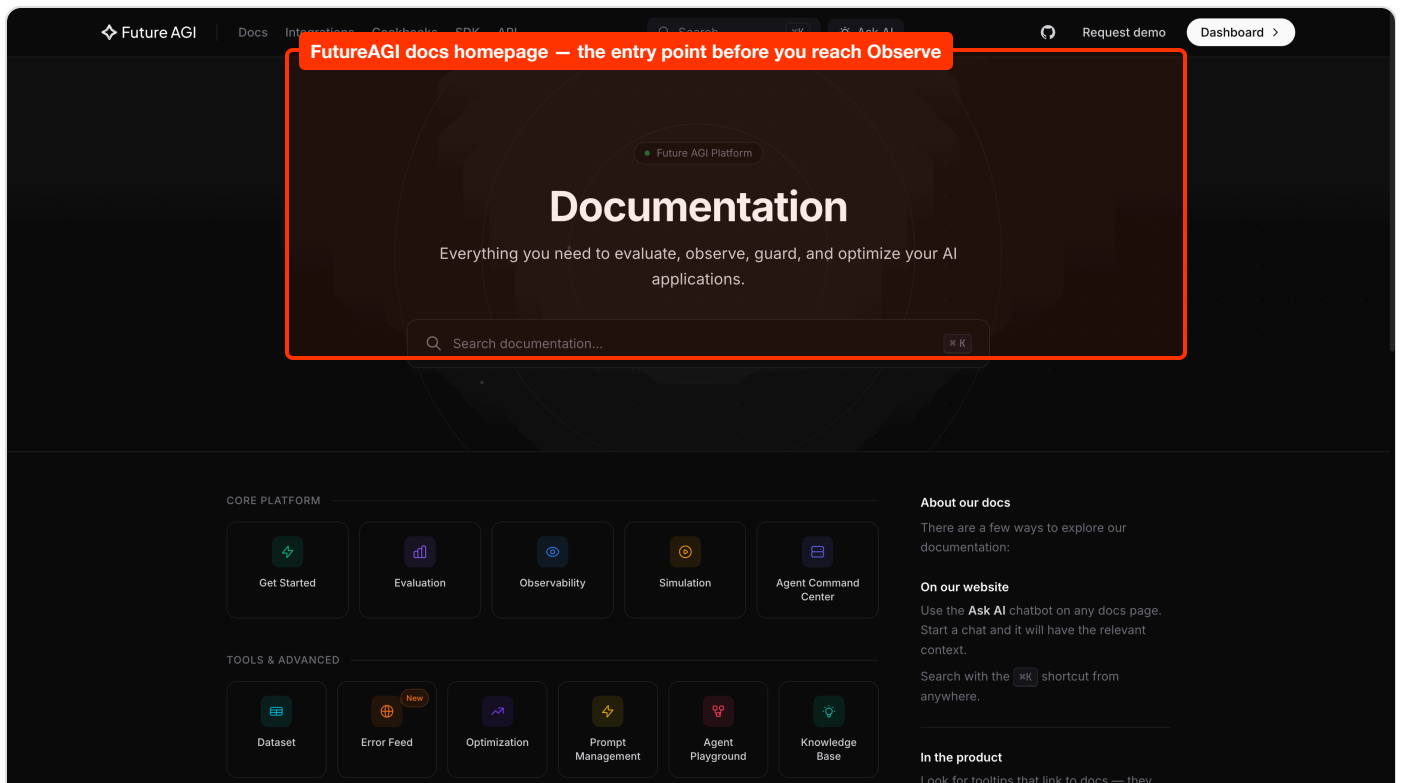
► Reference

Trace Filter Syntax · Dashboard Metric Definitions · Trace Export and Endpoints

► Troubleshooting (4 pages)

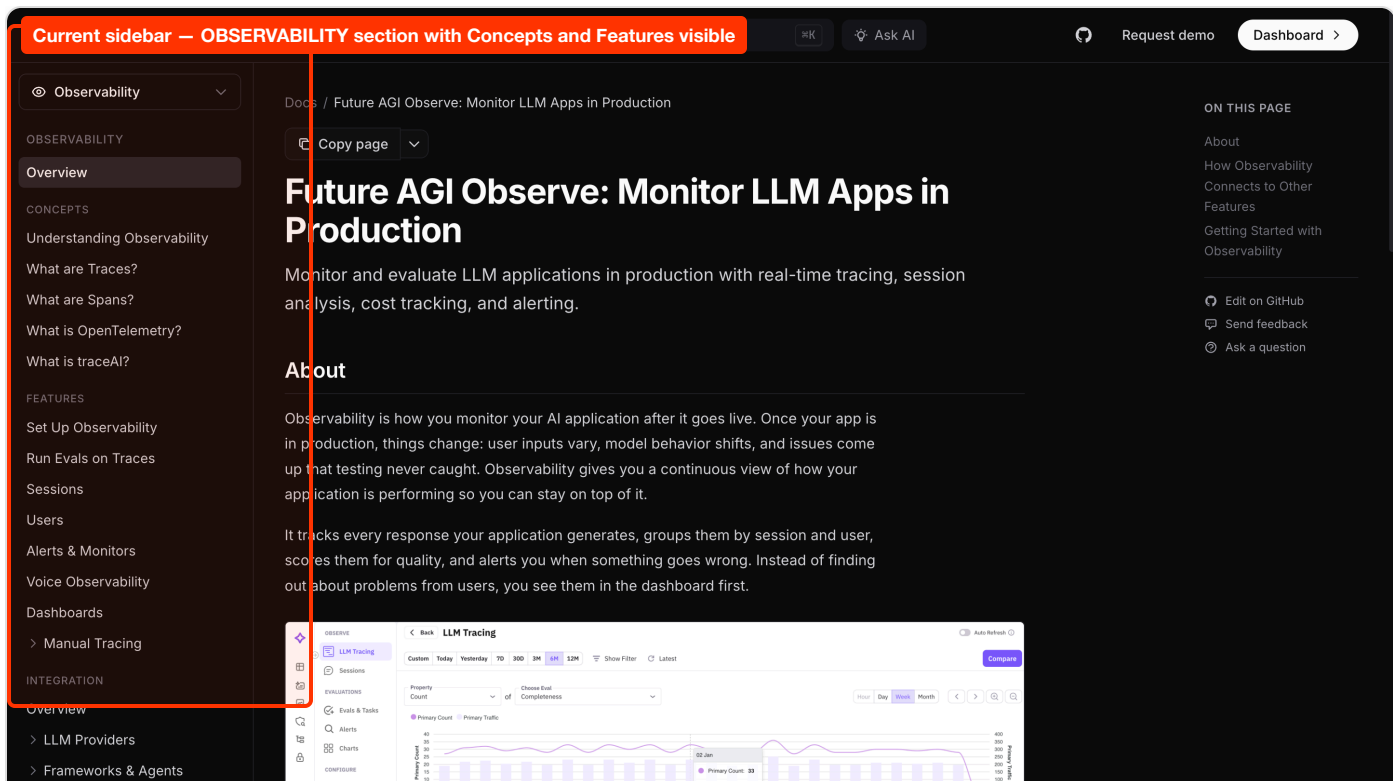
► Framework integrations

Overview · LLM Providers · Frameworks & Agents · Voice · Java



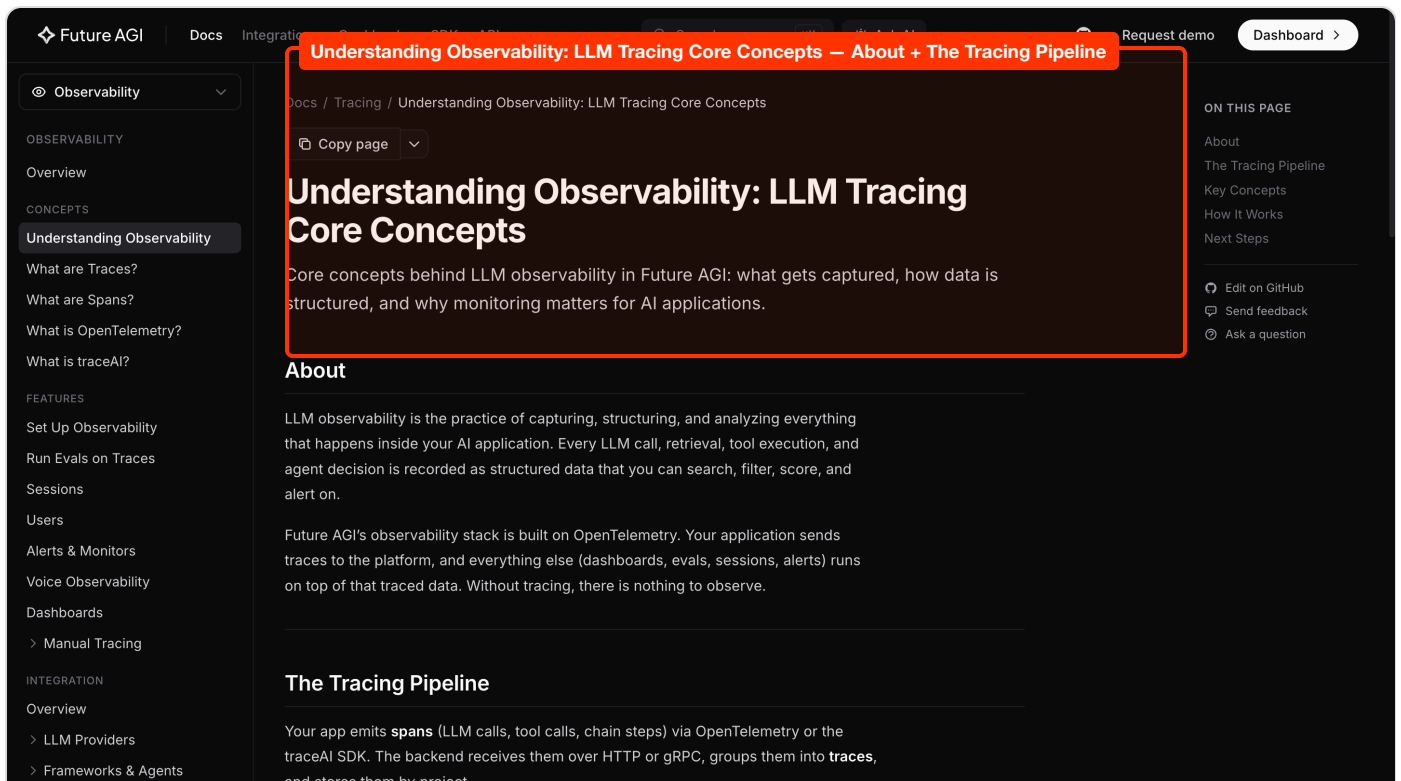
51 — FutureAGI docs homepage: this is what a user sees before they navigate to Observe. The overall docs homepage sets the first impression — and the question is whether it makes Observe feel like a first-class product the way PostHog's product selector or Langfuse's top-nav tabs do. This is the baseline we're improving against.

The homepage sets the tone for the whole docs experience. If a user lands here and can't immediately find Observe as a clear, prominent product entry point, they're already starting from behind. Every competitor we studied makes their observability product instantly findable from the docs root.



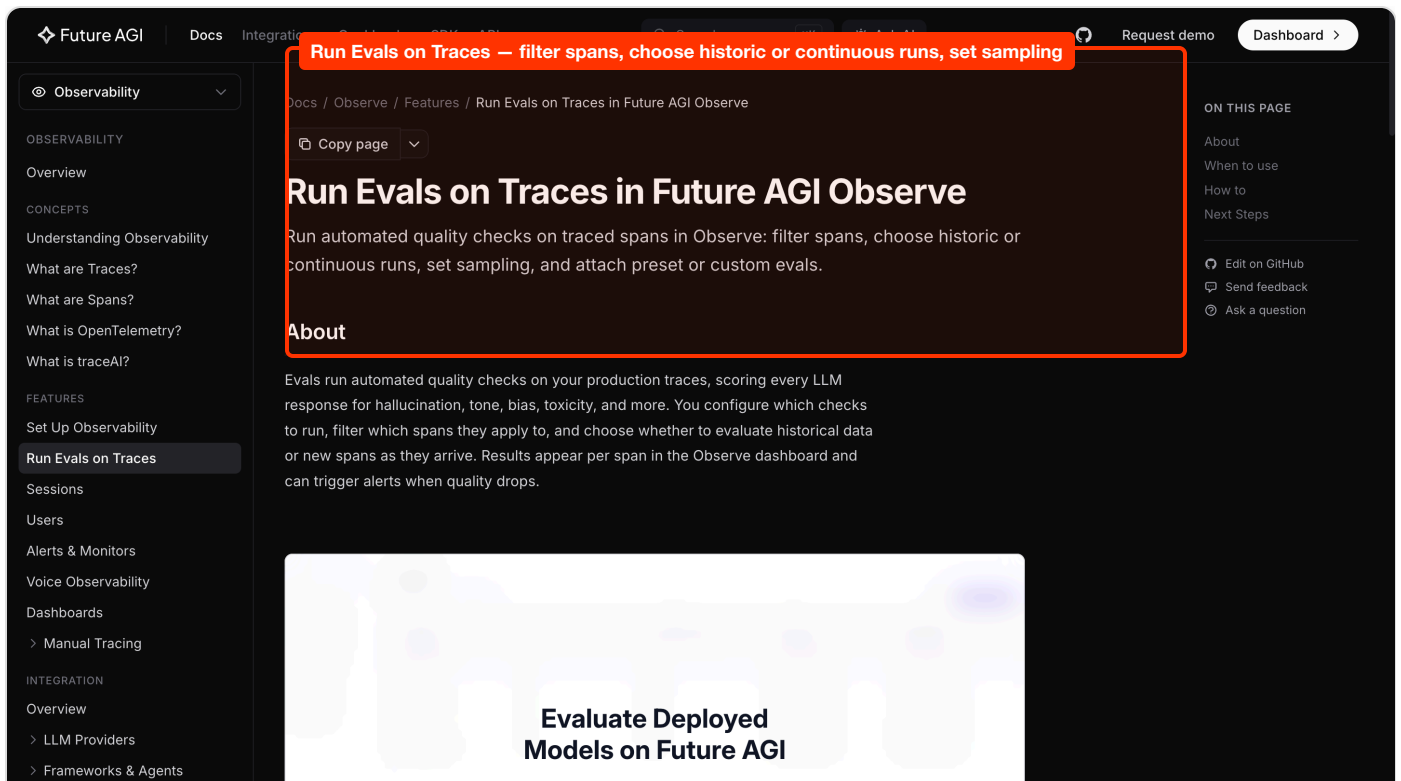
52 — FutureAGI Observe sidebar (live at docs.futureagi.com/docs/observe): the visible sidebar shows the OBSERVABILITY section: Overview, then CONCEPTS (Understanding Observability, What are Traces?, What are Spans?, What is OpenTelemetry?, What is traceAI?), then FEATURES (Set Up Observability, Run Evals on Traces, Sessions, Users, Alerts & Monitors, Voice Observability, Dashboards, Manual Tracing). No "Getting Started" entry is present. The Integration section (Framework integrations) is visible further down. Per `navigation.ts`, the full section order is: Overview → Concepts → Features → Reference → Troubleshooting → Framework integrations — meaning Troubleshooting is not last; Framework integrations follows it.

Walking this sidebar as a first-time user feels confusing. There's no "Getting Started" entry — you land straight on Overview/Concepts. The question-form labels ("What are Traces?", "What is OpenTelemetry?") read as FAQ entries, not navigation. The section ordering problem (Troubleshooting before Framework integrations) is confirmed in `navigation.ts` rather than visible in this single frame: Reference → Troubleshooting → Framework integrations, with Troubleshooting in the second-to-last slot when it should be last. Moving Framework integrations before Troubleshooting costs zero content work.



53 — FutureAGI tracing concepts page: the page at `docs.futureagi.com/docs/tracing/concepts` is titled "Understanding Observability: LLM Tracing Core Concepts — Core concepts behind LLM observability in Future AGI: what gets captured, how data is structured, and why monitoring matters for AI applications." The visible sections are "About" (defining LLM observability as capturing every LLM call, retrieval, tool execution, and agent decision) and "The Tracing Pipeline" (explaining that apps emit spans via OpenTelemetry or the traceAI SDK). The right-rail TOC lists: About, The Tracing Pipeline, Key Concepts, How It Works, Next Steps.

The concepts page establishes the vocabulary: LLM observability, spans, traces, OpenTelemetry, traceAI SDK. That's the right starting point. But compared to Braintrust's Observe page — which includes a "Why observe in Braintrust" motivational section, filter examples, and dashboard coverage all on one page — our concepts page stops at definition level. The gap is motivational depth and actionable next steps.



55 — FutureAGI filter surface (Run Evals on Traces page): the closest live page covering filter-level configuration is "Run Evals on Traces in Future AGI Observe" at docs.futureagi.com/docs/observe/features/evals. Its subtitle reads: "Run automated quality checks on traced spans in Observe: filter spans, choose historic or continuous runs, set sampling, and attach preset or custom evals." Note: the originally cited URL [/docs/observe/reference/trace-filter-syntax](https://docs.futureagi.com/docs/observe/reference/trace-filter-syntax) returns 404 — a dedicated Trace Filter Syntax reference page does not exist on the live site and has no source .mdx file in the current docs repo.

The Evals page surfaces filtering as part of eval configuration — "filter which spans they apply to" — but it's not a standalone filter syntax reference. This is the gap: our filtering capability exists (you can filter spans for evals, for alerts, for monitoring) but there's no single page that documents the filter model, the operators, and the field list. Braintrust's Common Filters page is the direct target.

Set Up Observability: When to use — First-time setup, Production monitoring, Debugging, Baseline for other features

- **First-time setup:** Get traces flowing into the Observe dashboard so you can start monitoring production LLM calls.
- **Production monitoring:** See latency, cost, and token usage for every LLM call in one place instead of scraping logs.
- **Debugging:** Tie a user report or failure to a specific trace and span so you can reproduce and fix issues.
- **Baseline for other Observe features:** Sessions, evals, user tracking, and alerts all require traces to be set up first.

How to

- 1. Install the packages**

Install the core instrumentation package and the framework instrumentor for your LLM provider.

```
Python JS/TS
```

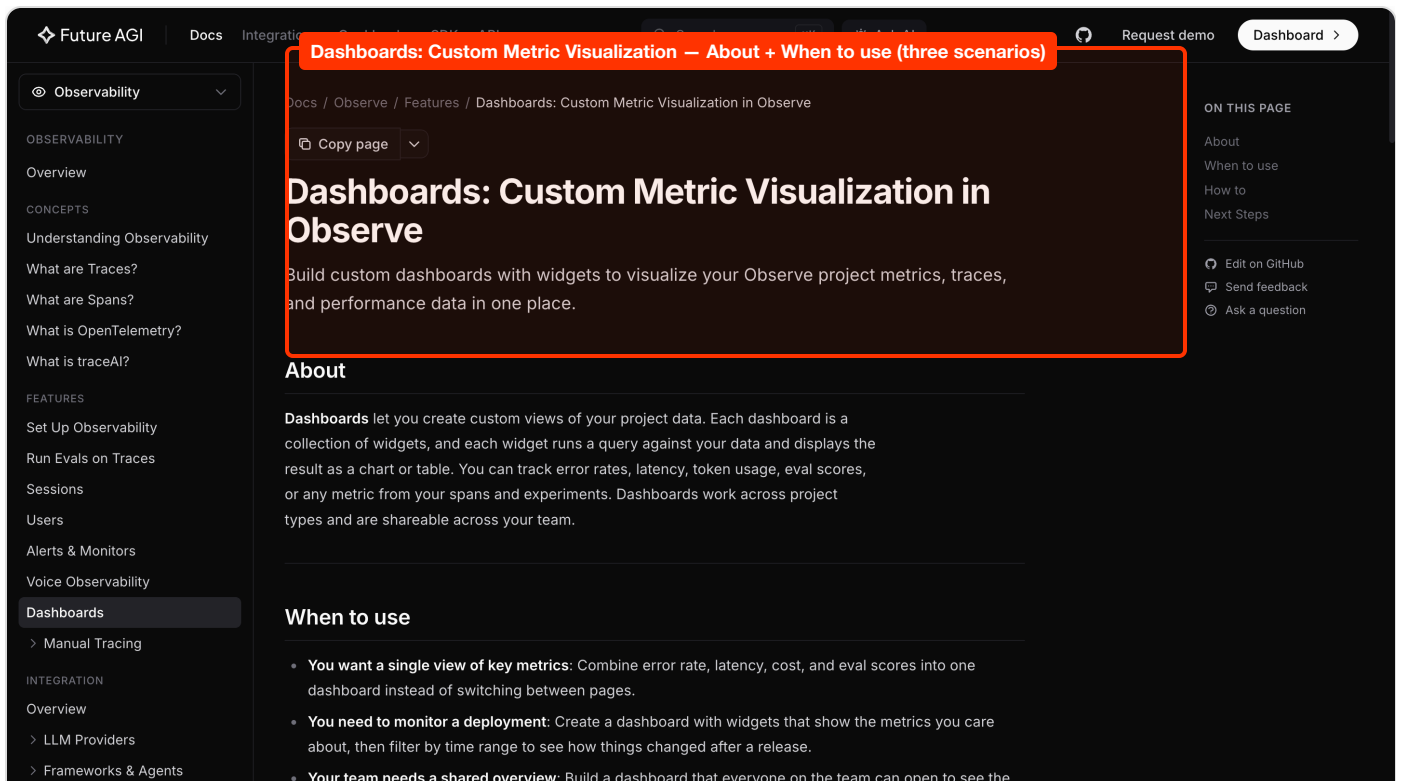
```
pip install fi-instrumentation-otel traceAI-openai
```
- 2. Configure your environment**

Set environment variables so the SDK can connect to Future AGI. Get your API keys from the [dashboard](#).

```
Python JS/TS
```

56 — FutureAGI Set Up Observability (tracing depth baseline): the closest live page to an LLM-tracing depth reference is "Set Up Observability with Future AGI Observe" at docs.futureagi.com/docs/observe/features/quickstart. The "When to use" section lists four scenarios: First-time setup (get traces flowing into Observe), Production monitoring (see latency, cost, and token usage per LLM call), Debugging (tie a user report to a specific trace and span), and Baseline for other features (Sessions, Evals, Users, Alerts all require traces). Note: the originally cited URL </docs/observe/features/llm-tracing> returns 404 — a dedicated LLM Tracing page does not yet exist on the live site.

The quickstart page covers the "when to use" framing well — four concrete scenarios rather than abstract capability claims. But it stops at the setup level. There's no per-span breakdown, no latency/cost column explanation, no filter-by-span-status example. The LLM Tracing depth page (in observe-two source but not yet deployed) is the gap we're filling.



54 — FutureAGI Dashboards page: the page at `docs.futureagi.com/docs/observe/features/dashboard` is titled "Dashboards: Custom Metric Visualization in Observe — Build custom dashboards with widgets to visualize your Observe project metrics, traces, and performance data in one place." The About section explains dashboards as collections of widgets (chart or table), each running a query against project data. The "When to use" section lists three scenarios: single view of key metrics, monitor a deployment, team shared overview.

The page covers what dashboards are and three reasons to use them — that's a reasonable foundation. But there's no step-numbered walkthrough: Create → Add Widget → Configure the metric → Share the URL. Mixpanel's Boards doc does exactly that. A user reading our Dashboards page can't come away knowing how to actually build one; they know what it is, not how to do it.

Synthesis

Four analytical artifacts derived from the evidence above. These are the outputs that feed the Execution PRD.

§4 The Content-Depth Rubric — "Feels Like Braintrust"

A qualitative rubric with 8 dimensions. Scores: 1 (shallow) → 5 (gold standard).

Dimension	Definition	Shallow (1–2)	Adequate (3)	Gold standard (4–5)
Conceptual "Why" Prose	Does the page explain why you'd do this, not just what it does?	Feature described as a list of capabilities only	One intro paragraph with motivation	A dedicated motivational section explains concrete business reasons (e.g., Braintrust's "Why observe in Braintrust" with specific feedback-loop bullets)
Worked Examples	Does the page show a concrete, end-to-end example?	No examples; abstract description only	One basic code snippet	Multiple examples covering the common case and at least one edge case, with expected output shown (e.g., Braintrust's filter accordions with copy-pasteable SQL expressions)
Code Completeness	Is code runnable as-is, or does it require inference?	Partial pseudocode or missing imports	Runnable but minimal; no error handling	Complete, copy-pasteable snippets with all imports present, inline parameter comments, and at least one complete end-to-end flow (e.g., Sentry's Options page with annotated <code>init()</code> call)
Parameter / Field Tables	Are all parameters documented with type, default, and description?	No parameter docs; must infer from code	Some parameters listed in prose	Full table with param name, type, default, description, and valid values for every option — no inference required (e.g., Sentry's Extended Configuration → Options page)
Annotated Screenshots	Do UI screenshots have labeled callouts pointing to specific elements?	No screenshots or raw unlabeled images	Screenshots with alt text but no callouts	Annotated screenshots with labeled callouts pointing to the specific UI elements being discussed — not generic screenshots (e.g., Braintrust's right-rail TOC callout, Sentry's step-numbered install flow)
Edge Cases & Gotchas	Does the page surface "what goes wrong" and how to avoid it?	No warnings or caveats	One "Note" callout	Dedicated troubleshooting callout or subsection per feature; common pitfalls named with exact error

Dimension	Definition	Shallow (1–2)	Adequate (3)	Gold standard (4–5)
				messages and specific fixes — not just "see Troubleshooting" (e.g., Langfuse's per-page Troubleshooting section with GitHub Discussions embedded)
Cross-linking	Does the page link to related concepts, next steps, and references?	No cross-links or only footer "Next" link	A few inline links	Inline links throughout the body; a "Next steps" section; links to related concepts and API reference — user never hits a dead end (e.g., Braintrust's Observe page linking to Filter, Dashboards, and Loop in one flow)
Progressive Disclosure	Does the page flow from overview → detail, not dump everything at once?	All information at same density level	Some sub-sections but no logical depth order	Clear depth progression: overview → concept → how-to → advanced → reference → troubleshooting — each section assumes the previous, none repeat it (e.g., PostHog AI Observability sidebar order)

Score Comparison — All 6 Subjects

Subject	Why Prose	Examples	Code	Param Tables	Annotated UI	Edge Cases	Cross- links	Disclosure	Avg
Braintrust	5	5	5	4	4	4	5	5	4.6
Langfuse	4	4	4	3	4	4	5	4	4.0
Sentry	3	4	5	5	3	4	4	4	4.0
PostHog	4	3	3	2	3	3	4	5	3.4
Mixpanel	3	3	2	2	4	2	4	4	3.0
FutureAGI (us)	2	2	2	1	2	1	2	2	1.8

§5 Sidebar Naming Pattern Sheet

Word-Count Distribution by Subject (Relevant Subtree)

Subject	1-word labels	2-word labels	3-word (conjunction)	Violations (3+ no conj)	Worst violation
Braintrust (Observe subtree)	~60%	~35%	~5% ✓	0	—
PostHog (AI Obs subtree)	~55%	~40%	~5% ✓	0	—
Sentry (Python subtree)	~50%	~45%	~5% ✓	0	—
Langfuse (Obs subtree)	~45%	~45%	~10% ✓	~1 ("Trace IDs & Distributed Tracing" = 5w but has &)	Medium
Mixpanel (Analysis subtree)	~65%	~30%	~5%	0 in core analysis	—
FutureAGI (current)	~30%	~30%	~15% ✓	~25% ✗	"Run Evals on Traces" (4w), "Understanding Observability" (2w but gerund-form)

Strong Short Labels to Adopt or Adapt

Label	Source	vs. Our Current	Action
Start here	PostHog, Langfuse	We have no entry point	Add as first item
Overview	All six	We have it ✓	Keep
Concepts	Braintrust, PostHog, Langfuse	We have it ✓	Keep, unify sub-pages
Traces / Spans	Langfuse, PostHog	We have "What are Traces?" ✗	Rename to noun form
Generations	PostHog	We don't have it	Consider adding
Dashboards	Braintrust, our current	We have it ✓	Keep
Alerts	PostHog, Mixpanel	We have "Alerts & Monitors" ✓	Keep (conjunction OK)
Troubleshooting	All six	We have it ✓ (but not last)	Move to last position
Tags	Langfuse	Missing from our features	Add as feature page
Sessions	All six	We have it ✓	Keep
Users	All six	We have it ✓	Keep
Sampling	Sentry, Langfuse	We don't have it	Consider adding

Verdict: Our 2/3-Word Rule

The 2/3-word rule (max 3 words, middle must be a conjunction) is consistent with best-in-class. Every competitor studied uses 1–2 word labels for 90%+ of their sidebar items within product subtrees. The outliers (Mixpanel's full sidebar, some Langfuse advanced entries) are in outer/admin sections, not the core product IA. We should apply the rule strictly within the Observe subtree.

Current violations to fix: "Run Evals on Traces" → "Trace Evals" · "Understanding Observability" → "Observability" · "Set Up Observability" → move its content into a new "Quick Start" sub-page under a new "Getting Started" section · "What are Traces?" → "Traces" · "What are Spans?" → "Spans" · "What is OpenTelemetry?" → "OpenTelemetry" · "What is traceAI?" → "traceAI"

§6 IA / Flow Map

Side-by-Side Top-Level Section Order (All Subjects)

#	Braintrust	PostHog (AI Obs)	Sentry (Python)	Langfuse (Obs)	Mixpanel (Analysis)	FutureAGI (now)
1	Introduction	Overview	Quick Start ← INSTALL	Overview	Reports	Overview
2	Quickstarts (2)	Start here	Features	Get Started	Boards	Concepts
3	Plans & limits	Installation	Configuration	Concepts	Users/Cohorts	Features
4	Instrument	Concepts	Integrations	Features	Session Replay	Reference
5	Observe	Evaluations	Data Management	SDKs	Features	Troubleshooting ← NOT LAST ✗
6	Annotate	Guides	—	Integrations ↗	—	Framework integrations ← SHOULD PRECEDE TROUBLESHOOTING
7	Evaluate	PostHog AI	Troubleshooting ← LAST	Troubleshooting ← LAST	—	—
8	Deploy	Resources	—	—	—	—
9	Admin	—	—	—	—	—
10	Best Practices	—	—	—	—	—

Most Intuitive Ordering Observed

PostHog's AI Observability IA is the most intuitive. It reasons: you need to understand what the product is (Overview), then get set up (Start here + Installation), then understand the concepts (Traces, Spans, etc.), then go deeper on feature-specific guides (Guides), then advanced AI-powered features (PostHog AI), then reference/support (Resources). Every section is a natural next step from the previous one.

The structural moves that make it feel intuitive:

- **Action-first entry** ("Start here" not "What is this")

- **Concepts after Setup** — you can run the code before you understand the theory
- **Guides in the middle** — task-focused, not reference-focused
- **Advanced/AI features after basics** — progressive disclosure
- **Resources last** — Troubleshooting, Changelog, Pricing never interrupt the main learning flow

Proposed Observe IA (NON-BINDING — for the Execution PRD to react to)

⚠ This is a First-Draft Proposal — NOT Final

The order below is derived from the evidence above. The Execution PRD finalizes this. Abhijai has the final call on structure.

PROPOSED OBSERVE IA — derived from evidence, NON-BINDING. Label rule: max 3 words; if 3 words, the middle must be a conjunction (and/or/&); no gerunds; no question forms.

Observe | |— Overview [1w] | |— **Getting Started** [2w — MISSING TODAY; the clear entry point] |
 |— Quick Start [2w — install + first trace in 5 min; content from current "Set Up Observability"] | |— Concepts [1w — unified "how the pieces relate" page] | |— Traces [1w — was "What are Traces?"] | |— Spans [1w — was "What are Spans?"] | |— Sessions and Users [3w ✓ — conjunction, keep] | |— OpenTelemetry [1w — was "What is OpenTelemetry?"] | |— traceAI [1w — was "What is traceAI?"] | |— Features [1w] | |— LLM Tracing [2w — keep] | |— Sessions [1w — keep] | |— Users [1w — keep] | |— Tags [1w — ADD: per-attribute page, modelled on Langfuse] | |— Dashboards [1w — keep] | |— Alerts and Monitors [3w ✓ — conjunction, keep] | |— Voice Observability [2w — keep] | |— Trace Evals [2w — was "Run Evals on Traces" (4w violation)] |
 |— Manual Tracing [2w — keep, 14 sub-pages] | |— Integrations [1w — moved before Reference; was "Framework integrations" at the end] | |— Overview [1w] | |— LLM Providers [2w] | |— Frameworks and Agents [3w ✓ — conjunction; spell out "and", not "&"] | |— Voice [1w] | |— Java [1w] | |— Reference [1w] | |— Filter Syntax [2w — was "Trace Filter Syntax"] | |— Dashboard Metrics [2w — was "Dashboard Metric Definitions"] | |— Export and Endpoints [3w ✓ — was "Trace Export and Endpoints"] | |— Troubleshooting [1w — LAST ✓] | |— No traces appear [3w ✓ — conjunction, keep] | |— Spans and Attributes [3w ✓ — was "Missing spans or attributes" (4w); noun+conjunction+noun form] | |— Dashboard numbers [2w — was "Dashboard numbers look wrong" (4w)] | |— Alerts not firing [3w ✓ — was "An alert did not fire" (5w, article-led)]

§7 Feature-Presentation Pattern Library

Pattern 1: Step-Numbered Flow (Mixpanel)

What it is: Breaking a feature into explicit numbered steps (Step 1, Step 2, Step 3...) where each step maps to a discrete UI action.

Screenshot evidence: Mixpanel Insights, Step 2 — "Choose Your Measurement." Each step has an anchor link, making it shareable and navigable independently.

When to use it: Any feature that requires sequential UI steps to configure or use. Our candidates: Dashboards (create → add widget → configure), Setting up Alerts, Manual Tracing setup.

Move to steal: Numbered H2 headers (## Step 1, ## Step 2) with anchor links — makes each step a shareable URL.

Pattern 2: Right-Rail Heading Tree (Sentry + Braintrust)

What it is: A sticky right-side panel that shows all H2/H3 headings on the current page, allowing jump-navigation without scrolling.

Screenshot evidence: Sentry Python "ON THIS PAGE" panel with Prerequisites → Install → Configure → Verify → Next Steps. Braintrust's equivalent shows the same for their Observe overview.

When to use it: Any page with 4+ major sections (most of our feature pages qualify). This is a site-level implementation — our docs framework should support it.

Move to steal: Ensure our docs framework exposes right-rail TOC on all content pages. Limit H2 headings to 5–8 per page so the TOC stays scannable.

Pattern 3: Per-Attribute Feature Pages (Langfuse)

What it is: Every filterable/configurable attribute gets its own full page: what it is, how to set it in the SDK, how to filter by it in the UI, and edge cases.

Screenshot evidence: Langfuse's Features → Essential section: Sessions, User Tracking, Environments, Tags, Metadata — each a full page.

When to use it: For every trace attribute that a user might want to filter by or configure. Our candidates: Sessions, Users, Tags (add this), Environment, Metadata, Cost Tracking.

Move to steal: Add per-attribute pages to our Features section. Each page follows: definition → SDK setup → UI filtering → edge cases → troubleshooting.

Pattern 4: Tutorial Cards on Overview Page (PostHog)

What it is: On the product Overview page, a grid of cards (Guide → Tutorial title → one-line description) that link directly to real integration guides.

Screenshot evidence: PostHog AI Observability Overview: "How to set up OpenAI observability," "How to set up Cohere," "How to set up Anthropic's Claude" — each a card.

When to use it: Product overview pages where users arrive with a specific integration goal (they already use OpenAI, or LangChain, etc.). Replaces the abstract "get started" with concrete "here's your path."

Move to steal: On our Observe Overview page, add integration tutorial cards for the top 5 frameworks (OpenAI, Anthropic, LangChain, LlamaIndex, CrewAI).

Pattern 5: Action-First Copy (PostHog + Braintrust)

What it is: Sidebar labels and page titles that start with a verb or use an action noun — telling you what you'll do, not what the feature is.

Screenshot evidence: PostHog: "Start here," "Start capturing LLM data," "Analyze LLM performance," "Link Session Replay." Braintrust: "Tracing quickstart," "Evaluation quickstart."

When to use it: Getting Started pages and Guide pages where the user's goal is to accomplish something, not understand something.

Move to steal: Add a new "Quick Start" sub-page under a "Getting Started" section (this is not a flat rename of "Set Up Observability" — it's a new entry point at a higher level). The existing "Set Up Observability" page content can become the body of Quick Start. Rename guide sub-pages to task form: "Add custom attributes" not "Custom Attributes."

§8 Baseline Audit — Our Observe Docs Gap List

Confirmed gaps against every competitor pattern studied. Severity: Critical = blocks the core user journey; High = significantly worse than best-in-class; Medium = notable but workable.

CRITICAL

No "Getting Started" entry point. We jump from Overview → Concepts with no intermediate "Quick Start" or "Start here" page. Every competitor (PostHog, Langfuse, Sentry, Braintrust) puts a low-friction entry action as the first item. First-time users don't know where to begin.

Fix: Add "Getting Started" section at position 2, containing a "Quick Start" page (install → first trace → see data in dashboard, in ~5 minutes).

CRITICAL

Troubleshooting is not last. Currently (per `navigation.ts`): Reference → Troubleshooting → Framework Integrations. Nikhil's requirement: Troubleshooting always last. Sentry and Langfuse explicitly place Troubleshooting last; no competitor puts it before the midpoint of their sidebar. Framework Integrations is a critical onboarding resource and should come before Troubleshooting.

Fix: Move Troubleshooting to the absolute end. Move Framework Integrations to position 4 (after Features, before Reference).

CRITICAL

Label-rule violations throughout. "Run Evals on Traces" (4 words, no conjunction), "Understanding Observability" (gerund form), "Set Up Observability" (3 words, no conjunction), "What are Traces?" (question form). These signal inconsistency and make the sidebar feel unpolished.

Fix: Apply the 2/3-word rule systematically. See §5 table for all replacements.

HIGH

No per-filter attribute pages. We have one "Trace Filter Syntax" reference page. Langfuse (the winner) has individual pages for every filterable attribute: Sessions, Users, Tags, Environments, Metadata, Log Levels, Full-Text Search. Users who want to learn "how to filter by session" have to read a full syntax reference page.

Fix: Add per-attribute pages in Features: Tags, Environments (if applicable), Log Levels, Metadata. Each follows: what it is → SDK setup → UI filter usage.

HIGH

Content depth is 1.8 / 5.0 on the rubric. No parameter tables on any feature page. No edge cases or "what can go wrong" callouts on most pages. Code snippets exist but are not complete (missing imports, no expected output). This is the most significant quality gap.

Fix: For each Features page, add: (1) a parameter/option table, (2) a complete runnable code example, (3) a "Common issues" section with specific error messages and fixes.

HIGH

No step-numbered flow for Dashboards. Our Dashboards page explains the feature but doesn't walk you through using it step-by-step. Mixpanel's Boards/Insights flow is exemplary here — numbered steps, each with its own anchor link.

Fix: Rewrite Dashboards page as a 4-step flow: Create → Add Widget → Configure Metric → Share URL.

HIGH

No unified "Core Concepts" page. We have individual pages for Traces, Spans, Sessions/Users, OpenTelemetry, traceAI — but no page that shows how they all relate to each other. Langfuse's Concepts page defines the hierarchy: Observation → Trace → Session. Users need to understand the data model before they can understand the features.

Fix: Add a Concepts overview page (before the individual concept sub-pages) that maps the entity hierarchy with a diagram.

MEDIUM

Question-form labels in Concepts sub-section. "What are Traces?", "What are Spans?", "What is OpenTelemetry?", "What is traceAI?" — the question form makes the sidebar feel FAQ-like rather than navigational. Every competitor uses noun forms.

Fix: Rename to noun forms: "Traces," "Spans," "OpenTelemetry," "traceAI."

MEDIUM

No on-page right-rail TOC. Our pages don't have the Sentry/Braintrust right-side heading tree. Long pages (Manual Tracing has 14 sub-pages' worth of content) require excessive scrolling.

Fix: Enable right-rail "On this page" TOC in the docs site theme for all content pages with 4+ H2 sections.

MEDIUM

No "Tags" feature page. Tags are a universal trace attribute across all six subjects. Langfuse has a full page. PostHog documents it. We don't surface tags as a first-class feature — despite the fact that our filter syntax supports tag-based queries.

Fix: Add "Tags" to Features, with: what tags are → how to set them in traceAI → how to filter by them in the UI.

Appendix — Source Links + Capture Date

Subject	URL Captured	Capture Date	Screenshots
Braintrust	braintrust.dev/docs + /observe + /observe/filter + /evaluate + /instrument + /reference	June 16, 2026	01–06
Mixpanel	docs.mixpanel.com/docs + /reports/insights + /boards + /data-governance/lexicon	June 16, 2026	11–15
PostHog	posthog.com/docs/ai-observability + /ai-observability/traces + /product-analytics	June 16, 2026	21–26
Sentry	docs.sentry.io/platforms/python/ + /tracing/ + /configuration/	June 16, 2026	31–34
Langfuse	langfuse.com/docs + /observability/overview + /observability/data-model + /features/token-and-cost-tracking	June 16, 2026	41–45
FutureAGI	docs.futureagi.com + /docs/observe + /docs/observe/features/llm-tracing + /dashboards + /trace-filter-syntax	June 16, 2026	51–56

Local Source

IA transcription source: `/Users/abhiжайsrivastava/Desktop/THE FOLDER/docs/src/lib/navigation.ts`

Page source directory: `/Users/abhiжайsrivastava/Desktop/THE FOLDER/docs/src/pages/docs/observe/**`

Screenshots saved to: `/Users/abhiжайsrivastava/Desktop/THE FOLDER/observe-revamp/analysis-assets/`